CS 246 — Object-Oriented Software Development

Vishesh Gupta

University of Waterloo
Winter 2018, Nomair Naeem

Contents

1 Course Outline **1**
- 1.1 Course Information . . . 1
- 1.2 Marking Scheme . . . 1
- 1.3 Modules . . . 2

2 Lecture 1: The Linux Shell and File System **3**
- 2.1 Module 1: The Linux Shell . . . 3
  - 2.1.1 Types of Shells . . . 3
  - 2.1.2 History of the Linux Shell . . . 3
- 2.2 The Linux File System . . . 4
- 2.3 Basic Commands . . . 5

3 Tutorial 1: Getting Set Up **6**
- 3.1 Course Infrastructure . . . 6
- 3.2 Logging In to the Student Server . . . 6
- 3.3 Course Directory and A0 . . . 7

4 Tutorial 2: Shell Commands, Streams, grep and Vim **8**
- 4.1 List of Commands . . . 8
- 4.2 Input and Output Streams . . . 8
- 4.3 grep and Regular Expressions . . . 9
- 4.4 Vim Commands . . . 9

5 Lecture 2: Redirection, Globbing, Pipes and egrep **11**
- 5.1 cat and Keyboard Signals . . . 11
- 5.2 Input and Output Redirection . . . 11
- 5.3 Behind the Scenes: Three Streams . . . 12
- 5.4 Wildcards and Globbing . . . 12
- 5.5 Pipes . . . 13
- 5.6 Embedded Commands . . . 13
- 5.7 egrep . . . 13

6 Lecture 3: Regular Expressions, Permissions, Variables and Scripts **14**
- 6.1 Regular Expressions (continued) . . . 14
- 6.2 File Permissions . . . 15
- 6.3 Shell Variables . . . 16
- 6.4 Shell Scripts . . . 16
  - 6.4.1 Arguments to a Script . . . 16

6.4.2 Status Codes . . . 17
6.4.3 Evaluating Conditions . . . 17

7 Lecture 4: Shell Scripting, Testing and Intro to C++ 18
7.1 Conditions with `[ ]` (continued) . . . 18
7.2 If Statements . . . 18
7.3 Loops . . . 19
7.3.1 While Loops . . . 19
7.3.2 For Loops . . . 20
7.4 Testing . . . 20
7.5 Introduction to C++ . . . 21

8 Lecture 5: Input/Output in C++ and Strings 23
8.1 Hello World Revisited . . . 23
8.2 The Three Standard Streams . . . 23
8.3 Reading Input and Failure . . . 24
8.3.1 Shortening the Loop . . . 24
8.4 C++ Strings . . . 26
8.5 I/O Manipulators . . . 26
8.6 Other Sources of Data . . . 27

9 Lecture 6: File and String Streams, Overloading, References 28
9.1 File Streams . . . 28
9.2 String Streams . . . 29
9.3 Default Arguments . . . 30
9.4 Function Overloading . . . 30
9.4.1 Operators Are Functions . . . 31
9.5 Pointers and const . . . 31
9.6 Structures in C++ . . . 31
9.7 Parameter Passing . . . 31
9.8 lvalue References . . . 32
9.8.1 Restrictions on References . . . 32
9.8.2 Pass by Reference . . . 33

10 Lecture 7: Pass by Reference, Dynamic Memory, Operator Overloading 34
10.1 Recap: How `cin » x` Works . . . 34
10.2 Pass by Value vs. Pass by Reference . . . 34
10.3 Dynamic Memory Allocation (Heap Allocation) . . . 35
10.3.1 Heap Arrays . . . 36
10.4 Returning Values . . . 36
10.5 Operator Overloading . . . 36
10.5.1 Overloading the I/O Operators . . . 37
10.6 The Preprocessor . . . 37
10.6.1 The Define Directive . . . 38

11 Lecture 8: Preprocessor, Separate Compilation and Make **39**
11.1 The Preprocessor (continued) . . . 39
11.1.1 Conditional Compilation . . . 39
11.1.2 Defined or Not . . . 40
11.2 Separate Compilation . . . 40
11.2.1 How to Compile . . . 41
11.3 Make . . . 41
11.3.1 Generalizing with Variables . . . 42
11.3.2 Letting g++ Track Dependencies . . . 42
11.4 Include Guards . . . 43

12 Lecture 9: Classes, Constructors and Member Initialization Lists **44**
12.1 Separate Compilation: Rules . . . 44
12.2 C++ Classes . . . 44
12.2.1 The `this` Pointer . . . 45
12.3 Initializing Objects . . . 45
12.3.1 Advantages of Constructors . . . 46
12.3.2 The Default Constructor . . . 46
12.4 Member Initialization Lists . . . 46

13 Lecture 10: Copy Constructor, Destructor and Copy Assignment **49**
13.1 Recap: Member Initialization List . . . 49
13.2 The Big Five . . . 49
13.3 The Copy Constructor . . . 50
13.3.1 When Is the Copy Constructor Called? . . . 51
13.4 Single-Parameter Constructors . . . 51
13.5 The Destructor . . . 51
13.6 The Copy Assignment Operator . . . 52
13.6.1 Why Check for Self-Assignment? . . . 52

14 Lecture 11: Copy-and-Swap, Rvalues and Move Semantics **54**
14.1 Recap and the Failing-`new` Problem . . . 54
14.2 The Copy-and-Swap Idiom . . . 54
14.3 Member Functions vs. Standalone Functions . . . 55
14.4 Tracing Copies . . . 56
14.5 Rvalues and Rvalue References . . . 56
14.6 The Move Constructor . . . 57
14.7 The Move Assignment Operator . . . 57
14.8 Copy/Move Elision (Preview) . . . 57

15 Lecture 12: Elision, Rule of Five, Const Methods and Encapsulation **59**
15.1 Copy/Move Elision . . . 59
15.2 Rule of Five . . . 60
15.3 Short Topic: Arrays of Objects . . . 60
15.4 Short Topic: Const Methods . . . 60
15.4.1 Mutable Fields . . . 61
15.5 Invariants and Encapsulation . . . 61
15.5.1 The `class` Keyword . . . 62

15.6 Encapsulating Node . . . 63

16 Lecture 13: Iterator Pattern, Friends and Static Members 65
16.1 Design Patterns . . . 65
16.2 The Iterator Pattern . . . 65
16.2.1 `auto` . . . 66
16.2.2 Range-Based For Loops . . . 66
16.3 Friends . . . 67
16.4 Accessors and Mutators . . . 67
16.4.1 I/O Operators and Friends . . . 67
16.5 Static Fields and Static Methods . . . 68
16.5.1 Static Member Functions . . . 68

17 Lecture 14: System Modelling, UML and Inheritance 70
17.1 System Modelling . . . 70
17.2 UML . . . 70
17.3 Relationship 1: Composition (owns-a) . . . 70
17.4 Relationship 2: Aggregation (has-a) . . . 71
17.5 Relationship 3: Inheritance (is-a) . . . 72
17.5.1 Inheriting Private Members . . . 73
17.6 Protected Visibility . . . 73

18 Lecture 15: Method Overriding, Virtual Methods and Polymorphism 75
18.1 Overriding . . . 75
18.2 Object Slicing . . . 76
18.3 Virtual Methods . . . 76
18.4 Destructors Revisited . . . 77

19 Lecture 16: Abstract Classes, Templates, Vectors and Exceptions 79
19.1 Pure Virtual Methods and Abstract Classes . . . 79
19.2 Templates . . . 80
19.2.1 A Template List with an Iterator . . . 81
19.3 The STL: Dynamic-Length Arrays . . . 81
19.3.1 Checked and Unchecked Access . . . 82
19.4 C++ Exceptions . . . 82

20 Lecture 17: Exceptions and Design Patterns 84
20.1 Exceptions (continued) . . . 84
20.1.1 Stack Unwinding . . . 84
20.1.2 Recovery in Stages . . . 85
20.1.3 What Can Be Thrown . . . 85
20.1.4 Exceptions for Control Flow? . . . 86
20.1.5 Good Practice . . . 86
20.1.6 Exceptions and Destructors . . . 86
20.2 Design Patterns (continued) . . . 86
20.3 Iterator Pattern Revisited . . . 87
20.4 The Observer Pattern . . . 88

21 Lecture 18: Observer and Decorator Patterns, Big Five Revisited 89
21.1 The Observer Pattern . . . 89
21.1.1 Example: A Horse Race . . . 90
21.2 The Decorator Pattern . . . 91
21.2.1 Example: Pizza . . . 92
21.3 The Big Five Revisited (with Inheritance) . . . 93
21.4 Assignment Through Base Class Pointers . . . 94

22 Lecture 19: Abstract Base Classes, Factory Method, Template Method, NVI, Maps 96
22.1 Partial and Mixed Assignment: The Fix . . . 96
22.2 The Factory Method Pattern . . . 98
22.3 The Template Method Pattern . . . 98
22.4 The Non-Virtual Interface (NVI) Idiom . . . 99
22.5 The STL `map` . . . 100
22.6 The Visitor Pattern: Double Dispatch . . . 100

23 Lecture 20: Visitor Continued, Compilation Dependencies and Pimpl 103
23.1 Visitor Recap . . . 103
23.2 Visitor, Second Use: Adding Operations to a Hierarchy . . . 103
23.3 Compilation Dependencies . . . 104
23.4 The Pimpl Idiom . . . 105
23.5 The Bridge Pattern . . . 106

24 Lecture 21: Exception Safety, RAII and Smart Pointers 108
24.1 The Problem . . . 108
24.2 Refactoring by Hand . . . 108
24.3 RAII . . . 109
24.4 `std::unique_ptr` . . . 109
24.4.1 A Basic Implementation . . . 110
24.4.2 Ownership . . . 111
24.5 `std::shared_ptr` . . . 111

25 Topic: Exception Safety Guarantees 113
25.1 Levels of Exception Safety . . . 113
25.2 An Example . . . 113
25.2.1 Achieving the Strong Guarantee . . . 114
25.3 Exception Safety and `vector` . . . 115
25.3.1 `emplace_back` and `noexcept` . . . 115
25.4 Exception-Safe `operator=` . . . 115

26 Topic: Coupling, Cohesion and MVC 117
26.1 Measures of Design Quality . . . 117
26.1.1 Coupling . . . 117
26.1.2 Cohesion . . . 117
26.2 Decoupling the Interface . . . 118
26.3 Model–View–Controller (MVC) . . . 118
26.4 Other Design Principles . . . 120

27 Topic: Template Functions, STL Iterators and Algorithms 121
27.1 Template Functions . . . 121
27.2 Iterator Categories . . . 122
27.3 The `<algorithm>` Library . . . 122
27.4 Function Objects . . . 123
27.5 Lambdas . . . 123
27.6 Iterator Adaptors . . . 124
27.7 More Containers . . . 124
27.8 Example . . . 124

28 Topic: Casting and Run-Time Type Information 126
28.1 C-Style Casts . . . 126
28.2 `static_cast` . . . 126
28.3 `reinterpret_cast` . . . 126
28.4 `const_cast` . . . 127
28.5 `dynamic_cast` . . . 127
28.5.1 Casting Smart Pointers . . . 127
28.6 Using RTTI: Polymorphic Assignment Revisited . . . 127
28.7 Is Dynamic Casting Good Style? . . . 128

29 Topic: How Virtual Methods Work (Vtables and Object Layout) 129
29.1 The Question . . . 129
29.2 Dynamic Dispatch Needs Run-Time Information . . . 129
29.3 Vtables and Vptrs . . . 130
29.4 Costs . . . 131
29.5 Object Layout and Inheritance . . . 131
29.6 Calling Virtual Methods in Constructors . . . 131

30 Topic: Multiple Inheritance 132
30.1 Basics . . . 132
30.2 Ambiguity . . . 132
30.3 The Diamond Problem . . . 133
30.3.1 Virtual Inheritance . . . 133
30.4 Object Layout . . . 134
30.5 When to Use Multiple Inheritance . . . 134

Chapter 1

Course Outline

January 4, 2018

1.1 Course Information

Instructor	Nomair Naeem
Office	DC 3111
Email	`nanaeem@uwaterloo.ca`
Office hours	Thursdays, 9:30–11:00 am

About the Instructor

Nomair Naeem is a CS advisor and instructor. He has (literally) begged to change what he teaches: he wants to teach compilers, but he teaches CS 246.

1.2 Marking Scheme

Component	Weight
Assignments	40%
Midterm (February 27)	20%
Final exam (2.5 hours)	40%

Assignments

Assignment	Weight
A0	required (0%)
A1	5%
A2	7%
A3	7%
A4	9%
A5 (final project)	12%

Assignment Submission

Assignments are submitted to **Marmoset**, which takes the most recent, on-time best score. Each assignment has public ("release") tests and private tests that are only run after the deadline.

Submission rule

You lose a mark from the assignment for using more than three submissions on release tests, so test locally first.

1.3 Modules

1. **Linux shell** (about 2 weeks): files, commands, redirection, pipes, regular expressions, permissions, shell scripts.
2. **C++** (about 10 weeks): C++ as a "better C", classes, the Big 5, encapsulation, inheritance, polymorphism, templates, the STL, exceptions.
3. **Tools**: `g++`, `make`, `gdb`, `valgrind`, version control with `git`.
4. **Software engineering**: UML, design patterns, coupling and cohesion, exception safety.

Summary

Three-part assessment (assignments, midterm, final); the course moves from the Linux shell into C++ and object-oriented design, finishing with design patterns and software-engineering principles.

Chapter 2

Lecture 1: The Linux Shell and File System

January 4, 2018

2.1 Module 1: The Linux Shell

Definition 2.1. A **shell** is an interface to the operating system. The OS talks to the hardware; the shell lets the user talk to the OS. With a shell we

1. manipulate files, and
2. run programs.

2.1.1 Types of Shells

Graphical shell. Windows, macOS and Linux all have one: point and click, drag and drop. Intuitive to use.

The big disadvantage of a graphical shell is that it is not very powerful: every feature has to be explicitly implemented by somebody. For example, suppose a folder contains a lot of files with the extension `.c` and we want to rename all of them to have extension `.cc`. Unless someone built a "bulk rename" button, we are stuck doing it one file at a time.

Command-line shell. We type commands at a *prompt* (basically a blinking cursor). The biggest disadvantage is the steep learning curve; the big advantage is that it is very powerful. You do not need someone to have "supported" a feature: you combine small commands to build the feature yourself.

DOS used to be the command-line shell on PCs, but it has gotten stale.

2.1.2 History of the Linux Shell

The Linux command-line shell is a descendant of the original Unix shell (1970s), called simply *the shell* and written by Stephen Bourne. It had a dozen or two commands. Over time many shells appeared:

- the Korn shell (`ksh`),

- the C shell (`csh`), which became the Turbo C shell (`tcsh`).

By then the original shell was called the *Bourne shell* (`sh`), and its successor became the **Bourne Again Shell** (`bash`), which is what we use in CS 246.

To check that you are running bash:

```
$ echo $0
-bash
```

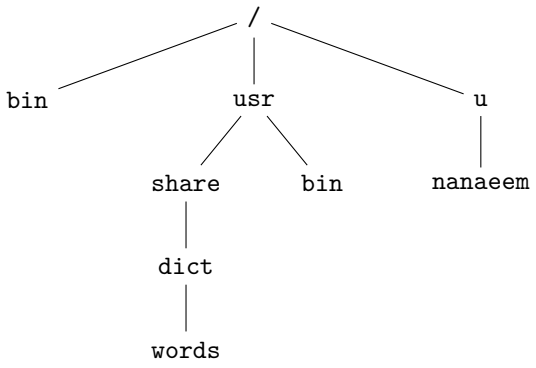
(`$0` is the name of the currently running program; the leading `-` means it is a login shell.)

2.2 The Linux File System

The file system is similar to the one on Windows. In Linux, there are files which can contain other files.

Definition 2.2. A **directory** is a file that can contain other files. Other than that, Linux does not really distinguish between files and directories.

The Linux file system is a **tree**. The root of the tree is a directory called the **root directory**, written `/`.



Definition 2.3. A **path** specifies the location of a file in the file system. An **absolute path** begins at the root directory, e.g. `/usr/share/dict/words`. A **relative path** starts at the current directory.

If `pwd` is `/Users/nanaeem`, then the absolute path `/Users/nanaeem/cs246/a1` names the same directory as the relative path `cs246/a1`.

Special directory names:

- `.` is the current directory,
- `..` is the parent directory, `../..` the grandparent,
- `~` is your home directory, and `~userid` is `userid`'s home directory.

2.3 Basic Commands

Command	Meaning
`clear`	clear the screen
`ls`	list the (non-hidden) files in the current directory
`ls -a`	list all files, including hidden ones (names starting with `.`)
`ls -l`	long listing (permissions, owner, size, date)
`pwd`	print the present working directory
`cd <path>`	change directory to `<path>`
`cd`	with no argument: go to your home directory
`cd -`	go back to the previous directory

The `-a` in `ls -a` is called an *option* (or *flag*); it modifies the behaviour of the command. Options can usually be combined, e.g. `ls -la`.

```
$ cd /Users/nanaeem/cs246/a1    # absolute path
$ cd ..                          # now in /Users/nanaeem/cs246
$ cd a1                          # relative path, back in a1
$ pwd
/Users/nanaeem/cs246/a1
```

Summary

- A shell is an interface to the OS; command-line shells trade a learning curve for power. We use `bash`.
- The file system is a tree rooted at `/`; directories are files that contain files.
- Absolute paths start at `/`; relative paths start at the current directory, with `.`, `..`, `~` as shortcuts.
- First commands: `clear`, `ls`, `pwd`, `cd`.

Chapter 3

Tutorial 1: Getting Set Up

January 3, 2018

Welcome to tutorial 1! (This surprisingly happened even before the first day of classes.)

3.1 Course Infrastructure

CS 246 course content is distributed through `git`, so we will need some `git` commands; they are explained as we go through the course.

A Linux environment is preferred (and recommended) for all work. All course material is available on the Linux student server, and assignments are tested there, so code should always be checked on the server before submitting.

3.2 Logging In to the Student Server

```
ssh username@linux.student.cs.uwaterloo.ca
# then enter the password for this account
```

If you do not have any clue what your `linux.student.cs` password was from CS 136, it can be reset through the CS Computing Facility account web page.

This method works directly on:

1. Linux,
2. macOS,
3. Bash on Ubuntu on Windows (Windows Subsystem for Linux; Windows 10 Creators Update or later). Setup instructions: `https://docs.microsoft.com/en-us/windows/wsl/about`.

Otherwise, download the **PuTTY** client (a free `.exe`) or **SuperPuTTY**.

Copying files

Files can be copied to and from the server with `scp`:

```
scp a1q1.cc username@linux.student.cs.uwaterloo.ca:~/cs246/a1/
scp username@linux.student.cs.uwaterloo.ca:~/cs246/a1/out.txt .
```

3.3 Course Directory and A0

The course account has a directory (`/u/cs246/`) that holds the current term's material; older terms' material is kept in older directories. A0 has been posted (Piazza has hints about where A0 lives each term).

Advice

Start the assignments early.

Summary

Use the Linux student server (via `ssh`/PuTTY), get course material with `git`, and complete A0 (which is required) early.

Chapter 4

Tutorial 2: Shell Commands, Streams, grep and Vim

January 10, 2018

4.1 List of Commands

Command	What it does
`cd`	change directory; with no argument goes home; `cd -` goes back to the previous directory
`clear`	clear the screen
`ls`	list files, e.g. `ls ..`, `ls -l`, `ls -a`
`touch`	create an empty file (or update a file's timestamp)
`pwd`	display the present working directory
`cat`	print the contents of a file
`head`	print the first 10 lines (`head -n 5` for the first 5)
`tail`	print the last 10 lines
`sort`	sort lines in lexicographical order
`uniq`	remove *adjacent* duplicate lines (so usually used after `sort`)
`shuf`	output a random permutation of the input lines
`tac`	like `cat` but prints the lines in reverse order
`wc`	count lines, words and characters

4.2 Input and Output Streams

Think of a program as a black box with one input stream and two output streams:

- **stdin**: by default the keyboard.
- **stdout**: by default the screen. stdout is *buffered*: output is collected and written in blocks, since writing character by character is expensive.
- **stderr**: by default also the screen. stderr is *not* buffered, so error messages appear immediately. Example: the nuclear reactor is about to explode — we want to see that message *now*, not when the buffer fills up.

We will solidify these concepts when we do I/O in C++.

```
g++ printer.cc -o print
./print > out.txt 2> err.txt   # stdout to out.txt, stderr to err.txt
./print &> all.txt             # all streams to all.txt
./print > all.txt 2>&1         # same: send stderr where stdout is going
```

Stream numbers

stdin is 0, stdout is 1, stderr is 2. `2>&1` redirects stderr to the same place as stdout; `&>` redirects both. Order matters: `2>&1` must come *after* `> file`.

Pipes. `cmd1 | cmd2` connects stdout of `cmd1` to stdin of `cmd2`.

Embedded commands. `$(cmd)` is replaced by the output of `cmd`, e.g. `egrep "$(cat pattern)" words.txt`.

4.3 grep and Regular Expressions

`grep` (and `egrep`, which uses extended regular expressions) prints the lines of its input that match a pattern:

```
egrep "int" wordcollection.txt
```

Symbol	Meaning
`^`	match the beginning of the line
`$`	match the end of the line
`.`	match any single character
`?`	the preceding item 0 or 1 times
`*`	the preceding item 0 or more times
`+`	the preceding item 1 or more times
`[ ]`	any one of the characters inside

4.4 Vim Commands

Keys	Effect
`i`	enter insert mode (`Esc` returns to normal mode)
`Esc :w`	write (save)
`Esc :wq`	write and quit
`Esc :x`	save (only if there are changes) and quit
`Esc :xa`	do `:x` on all open buffers/tabs
`Esc :q!`	quit without saving
`u`	undo
`Ctrl-r`	redo
`dd`, `yy`, `p`	delete line, copy (yank) line, paste
`/text`	search forward for `text`

Summary

Know the basic file/text commands, the three standard streams and how to redirect them (including `2>&1`), pipes and `$(...)`, basic regex for `egrep`, and enough Vim to edit, save and quit.

Chapter 5

Lecture 2: Redirection, Globbing, Pipes and egrep

January 9, 2018

5.1 cat and Keyboard Signals

`cat` stands for *concatenate*; with file arguments it displays the contents of the files one after another. With no arguments it echoes stdin to stdout.

- **Ctrl+C** — kill the running program.
- **Ctrl+D** — send the end-of-file (EOF) signal.

5.2 Input and Output Redirection

Definition 5.1. Output redirection sends the output produced by a program to a file: `command arg1 arg2 ... > filename`.

Input redirection sends the contents of a file to a program as its input: `command arg1 arg2 ... < filename`.

`cat sample.txt` vs. `cat < sample.txt`

The output is identical, but the mechanism is different:

- `cat sample.txt`: `cat` is given a filename, *cat* opens the file and displays its contents.
- `cat < sample.txt`: the *shell* opens the file and sends its contents to `cat`'s stdin; `cat` never sees a filename.

The difference is visible with `wc`:

```
$ wc sample.txt
 10  52 300 sample.txt
$ wc < sample.txt
 10  52 300
```

Input and output redirection can be combined (each used once):

```
cmd arg1 arg2 ... < inputfile > outputfile
cat -n < in.txt > out.txt    # like cp, but numbers the lines
```

5.3 Behind the Scenes: Three Streams

Every Linux process (`ls`, `cat`, a C/C++ program, ...) has three streams associated with it.

stdin (standard input)	stdout (standard output)	stderr (standard error)
Default: keyboard	Default: screen	Default: screen
Change with `<`	Change with `>`	Change with `2>`
	Buffered	Not buffered

Buffering: writing in blocks is faster than writing one character at a time (you will see why in CS 350). stderr is unbuffered so error messages show up immediately.

```
cmd >> out.txt              # append to the end of out.txt
cmd < in 1> out 2>&1        # stdout to out, stderr to the same place as stdout
```

`&1` means "wherever stream 1 (stdout) is currently going".

5.4 Wildcards and Globbing

```
ls *.txt
```

`*.txt` is a **globbing pattern**: `*` matches any sequence of characters, so this lists all files whose names end with `.txt`.

The *shell* intercepts the pattern:

1. it finds the files that match the pattern, and
2. replaces the pattern by those file names.

So `ls` actually receives `ls sample.txt sample2.txt`. Because the shell does the work, globbing works with any command:

```
cat *.txt
echo *.txt      # a cheap way to see what a pattern expands to
rm *.txt
rm -rf *        # careful!!
```

Other glob characters: `?` matches exactly one character, and `[abc]` matches one of the listed characters.

Quoting

Single quotes and double quotes suppress globbing: `echo "*.txt"` prints `*.txt`.

5.5 Pipes

We can send the output (stdout) of one program to the input (stdin) of another with a **pipe**: `cmd1 | cmd2`.

Example 5.2. How many words occur in the first 20 lines of `sample.txt`?

```
head -20 sample.txt | wc -w
```

Example 5.3. Files `words*.txt` contain lists of words, one per line. Print a duplicate-free list of all the words in all the files.

```
cat words*.txt | sort | uniq
```

`uniq` only removes *adjacent* duplicates, which is why we `sort` first.

5.6 Embedded Commands

Use `$(cmd)` to embed the output of a command inside another command:

```
echo "Today is $(date) and I am $(whoami)"
```

- Double quotes allow embedded commands (and variables) but suppress globbing; they also keep the result as a *single* argument.
- Single quotes suppress embedded commands as well: `echo 'Today is $(date)'` prints the text literally.
- Without quotes, each whitespace-separated word is a separate argument.

5.7 egrep

`egrep pattern file(s)` prints the lines that contain a substring matching the given pattern (a regular expression).

```
egrep cs246 index.shtml | wc -l          # number of lines mentioning cs246
egrep "cs246|CS246" index.shtml | wc -l  # either spelling
```

`|` in a regular expression means "or". Use double quotes around patterns containing characters that are special to the shell (`|`, `*`, `?`, ...), otherwise the shell interprets them first.

Summary

- `>`, `>>`, `<`, `2>`, `2>&1` redirect streams; the shell does the redirection, not the program.
- stdout is buffered, stderr is not.
- Globbing (`*`, `?`, `[...]`) is expanded by the shell; quotes suppress it.
- Pipes chain programs; `$(...)` embeds a command's output.
- `egrep` searches for lines matching a regular expression.

Chapter 6

Lecture 3: Regular Expressions, Permissions, Variables and Scripts

January 11, 2018

6.1 Regular Expressions (continued)

Last time: `egrep pattern file(s)`. For example

```
egrep "(cs|CS)246" index.html
```

This is *not* the same as `"(c|C)(s|S)246"` (which also matches `cS246` and `Cs246`); that one can also be written `"[cC][sS]246"`.

Pattern	Matches
`[abc]`	one character from the set, i.e. `(a|b|c)`
`[^abc]`	one character *not* from the set
`[a-z]`	one character in the range
`?`	0 or 1 occurrences of the preceding expression
`*`	0 or more occurrences of the preceding expression
`+`	1 or more occurrences of the preceding expression
`.`	any one character
`.*`	any number of any characters
`\.`	a literal `.` (backslash escapes a special symbol)
`^`	forces the match to start at the beginning of the line
`$`	forces the match to end at the end of the line

Examples:

- `"cs ?246"` matches `cs246` and `cs 246`.
- `"(cs)?246"` matches `cs246` or just `246`.
- `"^cs246"`: lines starting with `cs246`; `"^cs246$"`: lines containing exactly `cs246`.

Example 6.1. Print all words in `/usr/share/dict/words` that start with an `e` and have length 5.

```
egrep "^e....$" /usr/share/dict/words
```

Example 6.2. Lines of even length:

```
egrep "^(..)*$" filename
```

Anchors

When a pattern must *count* characters (exact length, exactly one occurrence, ...), anchor it with `^` and `$`. Otherwise `egrep` happily matches a substring of the line.

Example 6.3. List the file names in the current directory that contain exactly one `a`.

```
ls -a | egrep "^[^a]*a[^a]*$"
```

6.2 File Permissions

`ls -l` gives a long listing:

```
-rwxr-xr--  1 nanaeem  staff  2048 Jan 11 10:00 script.sh
```

The columns are: type and permissions, number of links, owner, group, size, last-modified date, and file name. The first character is `-` for an ordinary file and `d` for a directory. The next nine characters are three groups of three:

`rwx`	`r-x`	`r-`
user (owner)	group	other
the file's owner	users in the file's group	everyone else

Bit	On an ordinary file	On a directory
`r`	read the file	list the directory's contents (e.g. globbing)
`w`	modify the file	add/remove files in the directory
`x`	run the file as a program	enter the directory (`cd`)

Only the owner can change a file's permissions, using `chmod mode file(s)`. A mode consists of

1. an ownership class: `u`ser, `g`roup, `o`ther, `a`ll;
2. an operator: `+` (add), `-` (remove), `=` (set exactly);
3. permissions: `r`, `w`, `x`.

```
chmod a+r final.pdf      # give everyone read permission
chmod g-rw final.pdf     # remove read and write from the group
chmod o=rx final.pdf     # set others to exactly read/execute
chmod u+x myscript       # make a script executable by me
```

6.3 Shell Variables

```
x=1          # no spaces around = !  variables store strings
echo $x
echo ${x}    # recommended
```

When you *write* a variable, no `$`; when you *read* it, use `$`. It is good practice to use curly braces around variable names: `${x}abc` is unambiguous, `$xabc` refers to a variable named `xabc`.

- Double quotes allow variable expansion; single quotes suppress it: `echo '${x}'` prints `${x}`.
- Variables last for the current session only. To set something permanently, put it in `~/.bash_profile` (or `.bashrc`).

PATH

`echo ${PATH}` prints a list of directories separated by colons. When you type a command, the shell searches these directories (in order) for a program with that name.

```
PATH=mypath:${PATH}    # search mypath first
PATH=${PATH}:mypath    # search mypath last
```

6.4 Shell Scripts

Definition 6.4. A **shell script** is a text file containing a sequence of shell commands that can be executed as a program.

- The first line should be `#!/bin/bash`, the *shebang* ("hash bang") line. It tells the OS to run the script with bash.
- Give the script executable permission: `chmod u+x script`.
- Run it with `./script` (the current directory is usually not in `PATH`).

```
#!/bin/bash
date
whoami
pwd
```

6.4.1 Arguments to a Script

```
./script arg1 arg2 ...
#  $0     $1   $2
```

`$0` is the script's name, `$1`, `$2`, ... are its arguments, and `$#` is the number of arguments.

Example 6.5. Is a given word a valid word in the dictionary?

```
#!/bin/bash
egrep "^$1$" /usr/share/dict/words
```

6.4.2 Status Codes

Every Linux command sets a **status code**, stored in `$?`: `0` means success, non-zero means failure. For `egrep`, success means "found a match".

> To throw away output we don't care about, send it to `/dev/null`, a special file that discards everything written to it.

6.4.3 Evaluating Conditions

Conditions are evaluated with the `test` program, which can also be written as `[ ... ]`. The spaces inside the brackets are mandatory, because `[` is literally the name of a program and the rest are its arguments.

Example 6.6. A good password is a word that is *not* in the dictionary. Is a given word a good password?

```
#!/bin/bash
# The answer is the script's own status code.
egrep "^$1$" /usr/share/dict/words > /dev/null
[ $? -ne 0 ]
```

The status of a script is the status of its last command, so the script succeeds exactly when `egrep` failed to find the word.

`#` starts a comment. `[ -e basic ]` checks whether a file named `basic` exists.

Summary

- Regex: `[...]`, `[^...]`, `?`, `*`, `+`, `.`, `|`, grouping, and the anchors `^` and `$`.
- `ls -l` shows `rwx` permissions for user/group/other; `chmod` changes them.
- Variables: `x=1`, read with `${x}`; `PATH` is where commands are found.
- Scripts start with `#!/bin/bash`, receive `$1, $2, ...`, and communicate success through status codes; `[ ]` tests conditions.

Chapter 7

Lecture 4: Shell Scripting, Testing and Intro to C++

January 16, 2018

7.1 Conditions with `[ ]` (continued)

Recall that `[ cond ]` (the `test` program) evaluates a condition and sets its status code to 0 (true) or 1 (false). Output we do not care about is sent to `/dev/null`.

Test	True when
`[ s1 = s2 ]`, `[ s1 == s2 ]`	strings are equal
`[ s1 != s2 ]`	strings are different
`[ -z s ]`, `[ -n s ]`	string is empty / non-empty
`[ a -eq b ]`, `-ne`	integers equal / not equal
`-gt`, `-ge`, `-lt`, `-le`	integer $>$, $\geq$, $<$, $\leq$
`[ -e f ]`	file `f` exists
`[ -f f ]`, `[ -d f ]`	`f` is an ordinary file / a directory
`[ -r f ]`, `-w`, `-x`	`f` is readable / writable / executable
`[ c1 -a c2 ]`, `[ c1 -o c2 ]`	and / or
`[ ! c ]`	not

Strings vs. numbers

`=` and `!=` compare *strings*; `-eq`, `-lt`, ... compare *integers*. `[ 01 = 1 ]` is false but `[ 01 -eq 1 ]` is true. Always quote variables inside tests (`[ "${x}" = "abc" ]`) so an empty variable doesn't turn into a syntax error.

7.2 If Statements

```
if [ cond ]; then
    statements
elif [ cond ]; then
    statements
```

```
else
    statements
fi
```

The thing after `if` is really any command: the branch is taken if its status code is 0. So we can test a command directly, without looking at `$?`.

Example 7.1. Good-password checker, with output, and a usage message if called incorrectly.

```
#!/bin/bash
usage() {
    echo "Usage: $0 password" 1>&2     # error messages go to stderr
    exit 1
}

if [ $# -ne 1 ]; then
    usage
fi

if egrep "^$1$" /usr/share/dict/words > /dev/null; then
    echo "Not a good password"
else
    echo "Maybe a good password"
fi
```

Notes:

- `$#` is the number of arguments.
- `usage() { ... }` defines a shell *function*; inside a function, `$1`, `$2`, ... refer to the function's own arguments.
- `exit 1` ends the script with a non-zero (failure) status.
- `1>&2` sends the message to stderr.

7.3 Loops

7.3.1 While Loops

```
while [ cond ]; do
    statements
done
```

Example 7.2. Print the numbers from 1 to `$1`.

```
#!/bin/bash
x=1
while [ ${x} -le $1 ]; do
    echo ${x}
    x=$((x + 1))
done
```

Variables hold strings, so `x=x+1` would just store the string `x+1`. Arithmetic is done with `$((...))`.

7.3.2 For Loops

A `for` loop iterates over a list of words:

```
for x in a b c d; do
    echo ${x}
done
```

Example 7.3. Rename every `.cpp` file in the current directory to have a `.cc` extension (the task the graphical shell couldn't do in Lecture 1).

```
#!/bin/bash
for name in *.cpp; do
    mv "${name}" "${name%cpp}cc"
done
```

The glob `*.cpp` expands to the list of matching files. `${name%cpp}` is the value of `name` with the trailing `cpp` removed.

Example 7.4. How many times does the word `$1` appear in the file `$2`?

```
#!/bin/bash
x=0
for word in $(cat "$2"); do
    if [ "${word}" = "$1" ]; then
        x=$((x + 1))
    fi
done
echo ${x}
```

`$(cat "$2")` is deliberately unquoted so that it splits into one word per loop iteration.

Example 7.5. Payday is the last Friday of the month. On what day is this month's payday?

```
#!/bin/bash
answer() {
    if [ "$1" -eq 31 ]; then
        echo "This month: the 31st"
    else
        echo "This month: the ${1}th"
    fi
}
# cal prints a calendar; column 6 is Friday. Take the last non-empty one.
answer $(cal | awk '{print $6}' | egrep "[0-9]" | tail -1)
```

Generalizing to any month/year: pass `$1 $2` along to `cal` (`cal $1 $2`); with no arguments `cal` uses the current month.

7.4 Testing

Testing is an essential part of program development. It is *not* debugging: testing reveals the presence of bugs; it cannot prove their absence. Testing should happen *while* you write code, not only at the end.

- **Human testing**: code inspections, walkthroughs, explaining your code to someone else.

- **Machine testing**: running the program on inputs and checking the outputs.
- **Black-box testing**: tests are written from the specification only, without looking at the implementation. Cover various classes of input (numeric ranges, positive/negative), boundary cases (edges of ranges, empty input, first/last), multiple simultaneous boundaries (corner cases), intuition, and things that are clearly wrong (invalid input).
- **White-box testing**: tests are written with the code in view, so that every branch, loop and function is exercised (statement and branch coverage).
- **Performance testing**: is the program efficient enough?
- **Regression testing**: keep a test suite and re-run it every time the code changes, to make sure nothing that used to work is now broken.

A1 asks us to automate regression testing with a script. A minimal version: for each test stem listed in a suite file, run the program on `stem.in` and compare with `stem.out`.

```
#!/bin/bash
# usage: ./runSuite suite-file program
for stem in $(cat "$1"); do
    tmp=$(mktemp)
    "$2" < "${stem}.in" > "${tmp}"
    if ! diff "${tmp}" "${stem}.out" > /dev/null; then
        echo "Test failed: ${stem}"
    fi
    rm "${tmp}"
done
```

7.5 Introduction to C++

C++ was designed by Bjarne Stroustrup (early 1980s) as "C with classes". It is (almost) a superset of C, adding classes, references, operator and function overloading, templates, exceptions, a much bigger standard library, and more. In CS 246 we use the C++14 standard.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello world" << endl;
    return 0;
}
```

Differences from C that are already visible:

- `#include <iostream>` instead of `<stdio.h>`; I/O is done with *streams* (`cout`) and the operator « instead of `printf`.
- `using namespace std;` lets us write `cout` instead of `std::cout`.
- `main` must return `int`; `return 0` may be omitted in `main` (0 is returned by default).
- `endl` writes a newline and flushes the output buffer.

Compiling and running:

```
g++ -std=c++14 hello.cc            # produces a.out
./a.out
g++ -std=c++14 hello.cc -o hello   # name the executable
./hello
```

(On the student environment `g++14` is an alias for `g++ -std=c++14`.) C++ source files conventionally use `.cc` (or `.cpp`); headers use `.h`.

> In CS 246 we write C++, not C: use `new`/`delete` rather than `malloc`/`free`, and C++ streams and strings rather than `printf`/`scanf` and `char*`. Details start next lecture.

Summary

- `[ ]` tests strings (`=`, `!=`), integers (`-eq`, `-lt`, ...) and files (`-e`, `-f`, `-d`).
- `if/elif/else/fi`, `while ... do ... done`, `for x in list; do ... done`; arithmetic with `$((...))`; functions and `exit`.
- Black-box, white-box and regression testing; automate with scripts.
- C++ hello world, `iostream`, and compiling with `g++ -std=c++14`.

Chapter 8

Lecture 5: Input/Output in C++ and Strings

January 18, 2018

8.1 Hello World Revisited

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!" << endl;
    return 0;
}
```

```
g++ -std=c++14 hello.cc              # compiles to a.out
./a.out
g++ -std=c++14 hello.cc -o myprog    # compiles to myprog
```

8.2 The Three Standard Streams

Including `<iostream>` gives us three global variables:

Variable	Type	Use
`std::cout`	`ostream`	write to stdout with the *output operator* `«`: `cout « "Bla";`
`std::cin`	`istream`	read from stdin with the *input operator* `»`: `cin » x;`
`std::cerr`	`ostream`	write to stderr: `cerr « "Error message";`

As usual, `cout` is buffered and `cerr` is not.

Example 8.1. Add two numbers.

```cpp
#include <iostream>
using namespace std;

int main() {
    int x, y;
    cin >> x >> y;
    cout << x + y << endl;
}
```

8.3 Reading Input and Failure

`cin » x » y` waits to read two `int`s, *skipping whitespace* (spaces, tabs, newlines). Reading an `int` stops at the first character that cannot be part of an `int`.

- If a read fails (bad input or EOF), the program keeps going, but all subsequent reads automatically fail as well.
- So after a read we should check whether it succeeded: after a read, `cin.fail()` is true if the read failed.
- If the read failed because of EOF, then `cin.fail()` and `cin.eof()` are both true.

(Reminder: Ctrl+D sends EOF; Ctrl+C kills the process.)

Example 8.2. Read all `int`s from stdin and echo them to stdout, one per line. Stop when a read fails.

```cpp
#include <iostream>
using namespace std;

int main() {
    int i;
    while (true) {
        cin >> i;
        if (cin.fail()) break;
        cout << i << endl;
    }
}
```

8.3.1 Shortening the Loop

C++ defines an automatic conversion from `istream` (e.g. `cin`) to `bool`: `cin` converts to `true` if the last read did *not* fail. So `if (cin.fail())` can be written `if (!cin)`.

Next, recall that in C, if `int a = 21;` then `a » 3` is a *right bit shift*: shift the bits of `a` three places to the right (giving 2); « is the left bit shift. In C++, operators can have multiple meanings depending on the types of their operands; this is called **operator overloading**. When the left operand is an `istream`, » means "read".

Expressions produce values

`cin » i` is an expression. Its *side effect* is reading into `i`; its *value* is `cin` itself. That is why reads can be cascaded: `cin » x » y` is `(cin » x) » y`.

Putting it together, the loop shrinks step by step:

```
int main() {
    int i;
    while (true) {
        if (!(cin >> i)) break;   // read, then test the stream
        cout << i << endl;
    }
}

// ... and finally:
int main() {
    int i;
    while (cin >> i) {
        cout << i << endl;
    }
}
```

Example 8.3. Read all `int`s and echo them to stdout until EOF. Skip over non-integer input.

```
#include <iostream>
using namespace std;

int main() {
    int i;
    while (true) {
        if (cin >> i) {
            cout << i << endl;
        } else {                    // read failed
            if (cin.eof()) break; // ... because of EOF: done
            cin.clear();          // acknowledge the failure (lower the fail flag)
            cin.ignore();         // skip the offending character
        }
    }
}
```

`cin.clear()` must come before `cin.ignore()`, because a stream in the failed state ignores all operations.

Look-up exercise: ignoring all of "hello" in one go. `cin.ignore()` skips one character, so the input `hello` costs five trips around the loop. To skip the rest of the line:

```
#include <limits>
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

To skip just one whitespace-delimited word, read it into a `string` and throw it away.

8.4 C++ Strings

We have a `string` type in C++.

> *"What, no clapping? You do not feel the power of the string type."* — Nomair Naeem

In C, strings were simulated by null-terminated character arrays, which are

- error prone (forget the `\0`, overrun the array), and
- a memory-management burden (you must allocate enough space).

C++ has a `string` type. It is not built in; it comes from `#include <string>`. A C++ string grows as needed and manages its own memory.

```
string s = "hello";   // s is a C++ string; "hello" is a C string (const char*)
```

C strings are automatically converted to C++ strings, but not the other way around (use `s.c_str()` if you need a `const char*`).

Operation	C	C++
equality	`strcmp(s1, s2) == 0`	`s1 == s2`
comparison	`strcmp`	`!=`, `<`, `<=`, `>`, `>=`
length	`strlen(s)`	`s.length()`
concatenation	`strcat(s1, s2)`	`s1 + s2`, `s1 += s2`
individual characters	`s[0]`, ...	`s[0]`, ...

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;
    cin >> s;       // reads one whitespace-delimited word
    cout << s << endl;
}
```

To read a whole line (including spaces), use `getline(cin, s)`: it reads into `s` everything up to (and discarding) the next newline.

Look up `std::string` (e.g. on cppreference) for more operations: `substr`, `find`, `push_back`, `insert`, ...

8.5 I/O Manipulators

In C, `printf` uses format specifiers: `%d` (int), `%s` (`char*`), `%x` (int in hexadecimal). In C++ we use **I/O manipulators** instead:

```
int i = 95;
cout << i;          // prints 95 in decimal
cout << hex << i;   // prints 5f (95 in hex)
cout << dec << i;   // back to decimal: 95
```

`hex` doesn't print anything; its side effect is to change the internal state of the stream so that all subsequent integers print in hex. Manipulators like `hex` and `dec` are therefore called **sticky**.

Other useful manipulators (many need `#include <iomanip>`):

- `boolalpha` / `noboolalpha`: print `bool`s as `true`/`false` rather than `1`/`0`;
- `showpoint`, `setprecision(3)`, `fixed`: floating-point formatting;
- `setw(n)`, `setfill(c)`: field width and padding (`setw` is *not* sticky — it applies to the next item only);
- `skipws` / `noskipws`: whether `>>` skips whitespace.

8.6 Other Sources of Data

The stream abstraction can be used to read and write other sources of data. With `#include <fstream>` we get `ifstream` (input file stream) and `ofstream` (output file stream). Using them is absolutely no different from using `cin` and `cout` — details next lecture.

Summary

- `cin >> x` reads (skipping whitespace), `cout << x` and `cerr << x` write.
- A failed read sets the fail bit; check with `cin.fail()`, `cin.eof()` or just `if (cin >> x)`. Recover with `cin.clear()` then `cin.ignore()`.
- `cin >> x` evaluates to `cin`: this enables cascading and the idiom `while (cin >> i)`.
- Use `std::string`, not `char*`; `getline` reads lines.
- Manipulators (`hex`, `dec`, `boolalpha`, `setprecision`, ...) change stream state and are mostly sticky.

Chapter 9

Lecture 6: File and String Streams, Overloading, References

January 23, 2018

9.1 File Streams

The stream abstraction can be used on other sources of data:

```
#include <fstream>
// std::ifstream -- read from a file
// std::ofstream -- write to a file
```

```
// io/readFile.cc
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream filestream{"myfile.txt"};   // "myfile.txt" is a path
    string s;
    while (filestream >> s) {
        cout << s << endl;
    }
}
```

The file is opened when `filestream` is initialized.

Q. When is it closed?

A. We'll look at this in detail about five weeks from now: it is closed when the `filestream` variable goes out of scope (here, when `main` returns). No explicit `close` needed.

Constructor parameters exist for appending, binary mode and so on. The key point:

Streams are interchangeable

Anything we can do with an `istream` (`ostream`) variable such as `cin` (`cout`), we can do with an `ifstream` (`ofstream`) variable.

9.2 String Streams

We can also attach streams to strings, to read from or write to them:

```
#include <sstream>
// std::istringstream -- read from a string
// std::ostringstream -- write to a string
```

Building a string with output operations:

```
int low = 1, high = 100;
ostringstream ss;
ss << "Enter a value between " << low << " and " << high;
string s = ss.str();     // extract the accumulated string
```

Converting a string to a number, safely:

```
// io/getNum.cc
#include <iostream>
#include <sstream>
#include <string>
using namespace std;

int main() {
    int n;
    while (true) {
        cout << "Enter a number" << endl;
        string s;
        cin >> s;                  // always succeeds (other than at EOF)
        istringstream ss{s};
        if (ss >> n) break;
    }
    cout << "You entered " << n << endl;
}
```

Benefits:

- Anything the user types can be read into a string, so the read from `cin` only fails at EOF. (Exercise: as written, this loops forever if the user gives up and sends EOF; add a check.)
- If the read from `ss` fails, we don't need `clear` and `ignore`: we just make a fresh `istringstream` next time around.

This is often the preferred way to parse input.

```
// readIntsSS.cc: echo all ints, skipping non-ints
int main() {
    string s;
    while (cin >> s) {
        istringstream ss{s};
```

```
        int n;
        if (ss >> n) cout << n << endl;
    }
}
```

Homework (not for marks): compare `readInts5.cc` (the `clear/ignore` version from last lecture) with `readIntsSS.cc` on the input `hello123abc45`. The first skips one character at a time, so it eventually reads `123` and `45`. The second reads the whole word `hello123abc45` as one string; `ss >> n` fails at `h`, so nothing is printed. (On `123abc` the second version prints `123`.)

9.3 Default Arguments

```
void printFile(string name = "myfile.txt") {
    ifstream filestream{name};
    string s;
    while (filestream >> s) cout << s << endl;
}

printFile("other.txt");   // prints other.txt
printFile();              // uses the default argument: myfile.txt
```

Default arguments must always appear last:

```
void test(int x = 5, string str) {}          // error: won't compile
void foo(int x = 5, string str = "Hello") {} // OK
```

When you call a function and omit arguments, you can only omit the last one (or the last n):

```
foo(10, "World");   // OK
foo(10);            // OK, str = "Hello"
foo();              // OK, x = 5, str = "Hello"
foo("World");       // error: can't skip x
```

The default is filled in by the *caller* at compile time, which is why it belongs in the declaration (the header), not the definition.

9.4 Function Overloading

Q. Can two functions have the same name?

A. Not in C, but yes in C++. This is **function overloading**.

C++ allows functions with the same name as long as the number or types of parameters differ. Differing only in return type is not enough.

```
int neg(int x)   { return -x; }
bool neg(bool b) { return !b; }
```

The compiler picks the version based on the argument types, at compile time.

Q. Given `foo(int x = 5, string str = "Hello")` above, could we also implement `void foo(int x)`?

A. Declaring it is legal, but then the call `foo(10)` is *ambiguous* (both versions match equally well), so it won't compile. Default arguments and overloading must not overlap.

9.4.1 Operators Are Functions

```
int a = 21, b = 3;
a >> b;    // legal: bit shift
cin >> x;  // legal: read
```

C++ implements operators as functions. The expression `cin >> x` is a call to a function named `operator>>`, overloaded on the types of its operands.

9.5 Pointers and const

> *"Now we have a big NOTE in my NOTES."* — Nomair Naeem

Read declarations from right to left:

```
int n = 5, x = 7;

const int *p = &n;     // p is a pointer to a const int
p = &x;                // fine: p itself can change
*p = 10;               // illegal: can't change the int through p
n = 10;                // fine: n itself isn't const

int * const q = &n;    // q is a const pointer to an int
q = &x;                // illegal: q can't be re-pointed
*q = 10;               // fine
```

9.6 Structures in C++

```
struct Node {
    int data;
    Node *next;        // no need to write "struct Node *next"
};                     // don't forget the semicolon!

Node n = {3, nullptr}; // no "struct" here either
```

Do not use `NULL` or `0` for pointers in C++: C++11 introduced the keyword `nullptr`.

> *"Use **`nullptr`**, as I sometimes tend to go back to old habits."* — Nomair Naeem

9.7 Parameter Passing

```
void inc(int n) { n += 1; }

int x = 5;
inc(x);
cout << x << endl;   // prints 5
```

In C and C++, arguments are passed by value by default: `inc` gets a copy. In C we would pass a pointer:

```
void inc(int *n) { *n += 1; }

int x = 5;
inc(&x);
cout << x << endl;   // prints 6
```

This is why C needs `scanf("%d", &x)`. But in C++ we write `cin » x`, not `cin » &x`. How does `cin` change `x`?

9.8 lvalue References

C++ has a **reference** type, written `&`.

```
int x = 10;
int &z = x;          // z is an lvalue reference to x
int * const p = &x;  // z behaves much like this constant pointer...

z = 5;               // ... with automatic dereferencing: same as *p = 5
                     // (NOT *z = 5)
```

Definition 9.1. An **lvalue reference** is like a constant pointer with automatic dereferencing. It is another name (an *alias*) for the object it refers to.



If you take the address of `z`, you get the address of `x`; if you ask for `sizeof(z)` you get `sizeof(int)`. `z` has no identity of its own; it is only a name for `x`. (This is called *aliasing*.)

9.8.1 Restrictions on References

- References must be initialized: `int &z;` is an error.
- They must be initialized to an **lvalue**, i.e. something that denotes a storage location (has an address):

  ```
  int &z = x;      // OK
  int &w = y + y;  // error: y + y has no address
  int &v = 5;      // error
  ```

- Once initialized, a reference can never be made to refer to something else.
- You cannot create a pointer to a reference (`int &*p` is illegal; a reference to a pointer, `int *&r = p;`, is fine).

- You cannot create a reference to a reference.
- You cannot create an array of references.

9.8.2 Pass by Reference

```
void inc(int &n) { n = n + 1; }   // no pointer syntax needed

int x = 5;
inc(x);
cout << x << endl;   // prints 6
```

`n` is another name for `x`, so changing `n` changes `x`. This is how `cin » x` works: `operator»` takes `x` by reference.

Summary

- `ifstream`/`ofstream` and `istringstream`/`ostringstream` work exactly like `cin`/`cout`; files close automatically when the stream goes out of scope.
- Reading a string then parsing it with an `istringstream` is a robust way to handle bad input.
- Default arguments go last; overloading requires different parameter lists; operators are functions named `operator<op>`.
- Read `const` declarations right to left.
- Use `nullptr`. A reference is an alias: a constant pointer with automatic dereferencing, must be initialized to an lvalue.

Chapter 10

Lecture 7: Pass by Reference, Dynamic Memory, Operator Overloading

January 25, 2018

10.1 Recap: How `cin » x` Works

```
void inc(int &n) { n += 1; }
int x = 5;
inc(x);
cout << x << endl;   // prints 6
```

`cin » x` is really the function call `operator»(cin, x)`, where

```
std::istream &operator>>(std::istream &in, int &data);
```

Both parameters are references: `data` so that the read can change `x`, and `in` because streams cannot be copied (and so that we return the same stream, enabling cascading).

10.2 Pass by Value vs. Pass by Reference

```
struct ReallyBig { /* lots of fields */ };

void f(ReallyBig rb) { ... }
ReallyBig b;
f(b);       // a copy of b is created for f: expensive
```

To avoid the copy, in C we would pass a pointer to `b`. That works in C++ too, but references are nicer:

```
void g(ReallyBig &rb) { ... }        // rb is another name for b
g(b);                                // no copy; but g could change b

void h(const ReallyBig &rb) { ... }  // no copy, and
h(b);                                // h cannot change b
```

People often call `rb` a "const reference". Technically the correct reading is that `rb` is a *reference to const* (a reference itself can never be re-bound anyway).

Advice

Whenever possible, prefer to pass by reference to const for anything bigger than an `int`, unless the function needs to change the argument (pass by reference) or needs its own copy anyway (pass by value).

Another benefit of passing by reference to const:

```
void foo(int &x) { ... }
foo(5);      // illegal: if foo did x = 10, that would mean 5 = 10
foo(y + y);  // illegal: y + y is not an lvalue

void bar(const int &x) { ... }
bar(5);      // OK
bar(y + y);  // OK
```

C++ allows a reference to const to be initialized from a value that is not an lvalue: the compiler creates a temporary to hold the value and binds the reference to it. Since `bar` promises not to change `x`, nobody can notice.

10.3 Dynamic Memory Allocation (Heap Allocation)

In C:

```
int *p = malloc(n * sizeof(int));
...
free(p);
```

This works in C++, but in CS 246 we do not use it. In C++ we use `new` and `delete`. `new` is type-aware: it knows the size of the type, and (as we'll see) it calls the constructor.

```
struct Node {
    int data;
    Node *next;
};

Node *np = new Node;   // no need to say how much space is needed
...
delete np;             // free memory you heap-allocated earlier
```

It is always safe to `delete nullptr` (it does nothing).

```
Node n;
Node *np = new Node;
```



When `n` and `np` go out of scope, their stack memory is reclaimed. The heap Node continues to live on.

Memory leaks

Forgetting to deallocate heap memory causes a **memory leak**. Memory leaks are bad, and Marmoset tests run under `valgrind`: leaking even one byte gives you 0 on that test.

10.3.1 Heap Arrays

```
Node *narr = new Node[10];
...
delete [] narr;   // arrays must be freed with delete []
```

Memory allocated with `new` must be freed with `delete`; memory allocated with `new []` must be freed with `delete []`. Mixing them is undefined behaviour. (Why `[]` is needed is discussed at the end of CS 241: the runtime has to know how many destructors to run.)

10.4 Returning Values

```
Node getMeANode() {
    Node n;
    return n;
}
```

This returns by value, so (in principle) `n` is copied into the caller. That can be expensive — we come back to this later in the course (move semantics and copy elision).

```
// ------- DON'T DO THIS -------
Node *getMeANode() {
    Node n;
    return &n;     // n dies when the function returns!
}
```

The returned pointer is a **dangling pointer**: it points to memory that is no longer yours to access.

Rule

Never return a pointer or a reference to stack-allocated (local) memory.

Returning a pointer to heap memory is fine (but now the caller is responsible for deleting it):

```
Node *getMeANode() {
    Node *np = new Node;
    return np;
}
```

10.5 Operator Overloading

C++ lets us give meaning to C++ operators for types we create.

```
struct Vec {
    int x, y;
};
```

In C we would write a function `Vec add(Vec v1, Vec v2);`. In C++:

```
Vec operator+(const Vec &v1, const Vec &v2) {
    Vec v{v1.x + v2.x, v1.y + v2.y};
    return v;
}

Vec operator*(const Vec &v, int k) {
    return {v.x * k, v.y * k};   // the compiler knows to build a Vec
}

Vec operator*(int k, const Vec &v) {
    return v * k;                // so k * v also works
}

Vec v1{1, 2}, v2{3, 4};
Vec v3 = v1 + v2 * 2;            // {7, 10}: usual precedence applies
```

Each ordering of operands needs its own function: `v * k` and `k * v` are different calls.

10.5.1 Overloading the I/O Operators

```
struct Grade {
    int theGrade;
};

std::ostream &operator<<(std::ostream &out, const Grade &g) {
    out << g.theGrade << '%';
    return out;
}

std::istream &operator>>(std::istream &in, Grade &g) {
    in >> g.theGrade;
    if (g.theGrade < 0) g.theGrade = 0;
    else if (g.theGrade > 100) g.theGrade = 100;
    return in;
}

Grade g;
cin >> g;               // reads an int, clamped to [0, 100]
cout << g << endl;      // e.g. 85%
```

Returning the stream by reference is what makes `cout « g « endl` work.

10.6 The Preprocessor

The preprocessor is a program that runs *before* the compiler sees your code; it transforms your source text. Lines starting with `#` are **preprocessor directives**.

- `#include <file>`: copy and paste the contents of the header right here. Angle brackets are for standard library headers.
- `#include "file.h"`: same, for your own headers (searched for in the current directory first).

To see the output of the preprocessor:

```
g++ -E -P file.cc
```

10.6.1 The Define Directive

`#define VAR VALUE` creates a preprocessor variable; afterwards every occurrence of the token `VAR` is replaced by `VALUE`.

```
#define MAX 10
...
int array[MAX];   // the compiler sees: int array[10];
```

In C++ a `const int MAX = 10;` is usually preferable (it has a type and obeys scope). More uses of `#define` next lecture.

Summary

- Pass anything bigger than an `int` by reference to const; a reference to const can bind to temporaries.
- `new`/`delete` and `new[]`/`delete[]`; leaks are failures in this course.
- Never return pointers/references to locals.
- Overload operators by writing `operator+`, `operator«`, etc.; I/O operators take and return the stream by reference.
- The preprocessor handles `#include` and `#define` before compilation.

Chapter 11

Lecture 8: Preprocessor, Separate Compilation and Make

January 30, 2018

11.1 The Preprocessor (continued)

Last time: `#include`, and `g++ -E -P file.cc` to see the preprocessor output. Also `#define VAR VALUE`:

```
#define MAX 10
...
int arr[MAX];
```

11.1.1 Conditional Compilation

```
#define IOS 1
#define BBOS 2
#define OS IOS

#if OS == BBOS
    long long int key;     // only one of these lines
#elif OS == IOS
    int key;               // survives preprocessing
#endif
```

This requires manually editing the definition of `OS` to target IOS or BlackBerry OS. Better: define preprocessor variables from the command line.

```
g++ -DOS=2 file.cc      # same as putting "#define OS 2" at the top
```

A handy trick for "commenting out" a large block of code (which may itself contain `/* */` comments):

```
#if 0
    // block of code the compiler never sees
#endif
```

11.1.2 Defined or Not

`#define VAR` with no value defines `VAR` with the empty string as its value.

- `#ifdef VAR` — true if `VAR` has been defined.
- `#ifndef VAR` — true if `VAR` has *not* been defined.

```
// preprocess/debug.cc
int main() {
#ifdef DEBUGGING
    cerr << "setting x = 1" << endl;
#endif
    int x = 1;
    ...
}
```

```
g++ -DDEBUGGING debug.cc    # debug output on
g++ debug.cc                # debug output compiled out
```

11.2 Separate Compilation

Split a program into modules, each with:

- a **header** (`.h`) file: type definitions and declarations (prototypes) of functions — the *interface*;
- an **implementation** (`.cc`) file: the definitions (bodies) of the functions — the *implementation*.

```
// vec.h
struct Vec {
    int x, y;
};
Vec operator+(const Vec &v1, const Vec &v2);

// vec.cc
#include "vec.h"
Vec operator+(const Vec &v1, const Vec &v2) {
    return {v1.x + v2.x, v1.y + v2.y};
}

// main.cc
#include "vec.h"
int main() {
    Vec v{1, 2};
    v = v + v;
}
```

Definition 11.1. A **declaration** asserts that something exists (e.g. a function prototype). A **definition** gives its full details and allocates space (e.g. a function body). An entity can be declared many times but defined only once.

11.2.1 How to Compile

1. `g++ *.cc` — only works if every `.cc` file in the directory is part of the project.
2. List all the files: `g++ vec.cc main.cc`.
3. Compile separately. `g++ main.cc` on its own fails: by default `g++` tries to produce an executable, so it tries to *link*, and `operator+` is missing. Use `-c` to compile only:

```
g++ -std=c++14 -c vec.cc      # produces vec.o
g++ -std=c++14 -c main.cc     # produces main.o
g++ main.o vec.o -o myprog    # link the object files
```

Never compile headers

`.h` files are meant to be *included*, never compiled.

`-c` produces **object files** (`.o`). An object file contains the compiled binary code, plus a list of what the file *provides* and what it still *needs* from other files. Linking matches these up. `-o name` names the output.

The benefit: if only `vec.cc` changes, we only recompile `vec.cc` and re-link.

11.3 Make

The problem: keeping track of which files need to be recompiled after a change (and remembering that changing `vec.h` means recompiling both `.cc` files). The tool for this is **make**.

Create a file called `Makefile` that describes the layout of the program and the dependencies between files:

- `myprog` depends on `main.o` and `vec.o`;
- `vec.o` depends on `vec.cc` and `vec.h`;
- `main.o` depends on `main.cc` and `vec.h`.

```
# tools/make/example1/Makefile
# target: dependencies
#<TAB>recipe (the indentation MUST be a tab character)
myprog: main.o vec.o
    g++ main.o vec.o -o myprog

vec.o: vec.cc vec.h
    g++ -std=c++14 -c vec.cc

main.o: main.cc vec.h
    g++ -std=c++14 -c main.cc

.PHONY: clean
clean:
    rm *.o myprog
```

- `make` builds the first target (`myprog`); `make clean` builds the `clean` target. The first time, everything is compiled.
- If `vec.cc` changes, make sees that the timestamp on `vec.cc` is newer than `vec.o` ⇒ recompile `vec.o`. Now `vec.o` is newer than `myprog` ⇒ relink `myprog`. `main.o` is untouched.
- The `clean` target deletes all binaries so you can start fresh. `.PHONY` tells make that `clean` is not the name of a file.

11.3.1 Generalizing with Variables

```
# example2
CXX = g++
CXXFLAGS = -std=c++14 -Wall
OBJECTS = main.o vec.o
EXEC = myprog

${EXEC}: ${OBJECTS}
    ${CXX} ${CXXFLAGS} ${OBJECTS} -o ${EXEC}

main.o: main.cc vec.h
vec.o: vec.cc vec.h

.PHONY: clean
clean:
    rm ${OBJECTS} ${EXEC}
```

For a rule of the form `x.o: x.cc a.h b.h`, we can leave out the recipe: make infers `${CXX} ${CXXFLAGS} -c x.cc -o x.o`. (`-Wall` turns on all warnings — always use it.)

11.3.2 Letting g++ Track Dependencies

Keeping the dependency lists correct by hand is error prone. `g++` can figure them out:

```
g++ -MMD -c vec.cc
# -c   produces vec.o
# -MMD also produces vec.d containing:  vec.o: vec.cc vec.h
```

```
# example3 -- the general-purpose Makefile
CXX = g++
CXXFLAGS = -std=c++14 -Wall -MMD
OBJECTS = main.o vec.o
DEPENDS = ${OBJECTS:.o=.d}
EXEC = myprog

${EXEC}: ${OBJECTS}
    ${CXX} ${CXXFLAGS} ${OBJECTS} -o ${EXEC}

-include ${DEPENDS}

.PHONY: clean
clean:
    rm ${OBJECTS} ${EXEC} ${DEPENDS}
```

`-include ${DEPENDS}` pastes in the generated `.d` files (the `-` means "don't complain if they don't exist yet"). For a new project, only `OBJECTS` and `EXEC` need to change.

11.4 Include Guards

Suppose `main.cc` includes `vec.h` and also includes some other header (say `linalg.h`) which itself includes `vec.h`:

```
$ g++ -c main.cc
error: redefinition of 'struct Vec'
```

The struct was pasted in twice. The fix is an **include guard**:

```
// vec.h
#ifndef VEC_H
#define VEC_H
struct Vec {
    int x, y;
};
Vec operator+(const Vec &v1, const Vec &v2);
#endif
```

The first time the file is included, `VEC_H` is not defined, so it gets defined and the contents are kept. Every later inclusion sees `VEC_H` already defined and skips everything.

Rule

Always put include guards in all `.h` files.

Summary

- `#if`/`#ifdef`/`#ifndef` and `-DVAR=VALUE` give conditional compilation.
- Split code into headers (interface) and `.cc` files (implementation); compile each with `-c` and link the `.o` files.
- `make` rebuilds only what is out of date, based on timestamps; use variables and `-MMD` to get a reusable Makefile.
- Every header gets an include guard.

Chapter 12

Lecture 9: Classes, Constructors and Member Initialization Lists

February 1, 2018

12.1 Separate Compilation: Rules

Last time we fixed the "redefinition of struct Vec" error (`lectures/separate/example3`) with an include guard (`lectures/separate/example4`). The rules:

1. All `.h` files should have an include guard.
2. Never compile `.h` files.
3. Never `#include .cc` files.

No `using namespace std` in headers

Never put `using namespace std;` in a header file: every file that includes your header would be forced to have it too (possibly causing name clashes). In a header, refer to `std::string`, `std::istream`, `std::ostream`, etc. by their full names.

12.2 C++ Classes

The big OO innovation: we can put functions inside a struct.

```
// student.h
#ifndef STUDENT_H
#define STUDENT_H
struct Student {
    int assns, mt, final;
    float grade();
};
#endif

// student.cc
#include "student.h"
float Student::grade() {
```

```
    return assns * 0.4 + mt * 0.2 + final * 0.4;
}

// main.cc
Student billy{80, 50, 75};
cout << billy.grade() << endl;   // 70
```

`::` is the **scope resolution operator**. `Student::grade` means "the function `grade` in the scope of the class `Student`".

Definition 12.1.

- A **class** is (essentially) a struct type that can contain functions. In C++ every struct is a class.
- An **object** is an instance of a class: a value of type `Student` is a `Student` object; `billy` is an object.
- A function inside a class is a **member function** or **method**.

A method can only be called on an object of its class, and has access to the fields of the object on which it was called. In `grade`, `assns` means `billy.assns` when we call `billy.grade()`.

12.2.1 The `this` Pointer

Every method has a hidden parameter called `this`, which points to the object on which the method was called. In `billy.grade()`, `this == &billy`. So the method above is really

```
float Student::grade() {
    return this->assns * 0.4 + this->mt * 0.2 + this->final * 0.4;
}
```

> *"I like to use `this` when I need to use `this`."* — Nomair Naeem

(`this` is a constant pointer: you cannot re-point it.)

12.3 Initializing Objects

`Student billy{80, 50, 75};` uses C-style field-by-field initialization. This is limited: we cannot validate the values, compute anything, or allocate memory. In C++, you can write special methods called **constructors** (ctors) to initialize objects.

```
struct Student {
    int assns, mt, final;
    Student(int assns, int mt, int final);
    float grade();
};

Student::Student(int assns, int mt, int final) {
    this->assns = assns < 0 ? 0 : assns;   // we can sanity-check now
    this->mt = mt < 0 ? 0 : mt;
    this->final = final < 0 ? 0 : final;
}

Student billy{80, 50, 75};   // calls the constructor
```

A constructor has the same name as the class and no return type. Here the parameter names shadow the field names, so we need `this->` to reach the fields.

- `Student billy{80, 50, 75};` — stack object, calls the ctor with the three arguments.
- `Student *pBilly = new Student{80, 50, 75};` — heap object, same ctor.
- `Student billy = Student{80, 50, 75};` also works.

The `{...}` syntax is **uniform initialization** (C++11), and can be used for all types: `int x{5};`, `string s{"hello"};`. If a class has a ctor, `{...}` calls it; otherwise (for plain structs) it does field-by-field initialization.

12.3.1 Advantages of Constructors

Constructors are functions, so we get default arguments, overloading and sanity checks:

```
struct Student {
    int assns, mt, final;
    Student(int assns = 0, int mt = 0, int final = 0);
};

Student s1{60, 70};   // final = 0
Student s2;           // all zero: calls Student(0, 0, 0)
```

12.3.2 The Default Constructor

Every class comes with a **default constructor** (a ctor with no arguments) if you don't write any constructor. It default-constructs every field that is an object, and leaves primitive fields (`int`, pointers) *uninitialized*.

You lose the free default ctor

As soon as you write *any* constructor, the compiler no longer provides the default constructor.

```
struct Vec {
    int x, y;
    Vec(int x, int y) : x{x}, y{y} {}
};
Vec v;          // error: no default ctor
Vec w{1, 2};    // OK
```

12.4 Member Initialization Lists

Consider a class with a constant or reference field:

```
int z;
struct MyStruct {
    const int myConst;   // must be initialized
    int &myRef;          // must be initialized
};
```

We can't give them default values in the class (every object could need a different value), and we can't assign to them in the ctor body. Why? Because of how objects are built.

Steps of object construction

1. Space is allocated (stack or heap).
2. Fields are initialized: each field that is an object has its constructor called (default ctor, unless told otherwise).
3. The constructor body runs.

By the time the body runs, the fields have already been initialized, so `myConst = 5;` in the body would be an assignment, not an initialization. To hook into step 2, use the **member initialization list** (MIL):

```
struct Student {
    const int id;          // a student's id never changes
    int assns, mt, final;
    Student(int id, int assns, int mt, int final);
};

Student::Student(int id, int assns, int mt, int final)
    : id{id}, assns{assns}, mt{mt}, final{final} {}
```

In `id{id}`, the name outside the braces is always the *field*, and the expression inside the braces is looked up normally (so here it is the parameter). No `this->` needed.

Facts about the MIL:

- You *must* use it to initialize `const` fields, reference fields, and object fields with no default constructor.
- It can be used for all fields.
- Fields are initialized in the order they are *declared in the class*, regardless of the order in the MIL (compile with `-Wall` to be warned about mismatched orders).
- It can be more efficient than assigning in the body: for an object field like a `string`, assigning in the body means it is first default-constructed and then assigned (two steps), whereas the MIL constructs it directly with the right value (one step).

You can also give fields default values in the class (C++11); the MIL overrides them:

```
struct Vec {
    int x = 0, y = 0;
    Vec() {}                          // x = y = 0
    Vec(int x) : x{x} {}              // y = 0
    Vec(int x, int y) : x{x}, y{y} {}
};
```

Summary

- Include guards in headers, never compile headers, never include `.cc` files, no `using namespace std` in headers.

- Classes are structs with methods; methods get a hidden `this` pointer; `Class::method` defines a method outside the class.
- Constructors initialize objects; writing any ctor removes the free default ctor.
- Construction: allocate, initialize fields (MIL), run body. Use the MIL for `const`/reference fields and fields without default ctors; fields are initialized in declaration order.

Chapter 13

Lecture 10: Copy Constructor, Destructor and Copy Assignment

February 6, 2018

13.1 Recap: Member Initialization List

- must be used to initialize `const` and reference fields;
- can be used for all fields;
- initialization occurs in declaration order;
- can be more efficient than initializing in the ctor body.

```
Student::Student(int id, int assns, int mt, int final)
    : id{id}, assns{assns}, mt{mt}, final{final} {}
```

The name outside the `{}` is the field; the expression inside the `{}` is the value.

13.2 The Big Five

```
Student billy{60, 70, 80};
Student bobby{billy};      // or: Student bobby = billy;
```

Objects are copied by calling the **copy constructor**, and you get one for free. Every class comes with:

1. a default constructor (only if you write no ctor),
2. a copy constructor,
3. a destructor,
4. a copy assignment operator,
5. a move constructor (C++11),

6. a move assignment operator (C++11).

Items 2–6 are called the **Big Five**.

> *"There will be a question about them, and I have already made that question an hour ago."* — N. Naeem

13.3 The Copy Constructor

The free copy ctor does a field-for-field copy. For `Student` it is equivalent to:

```
Student::Student(const Student &other)
    : assns{other.assns}, mt{other.mt}, final{other.final} {}
```

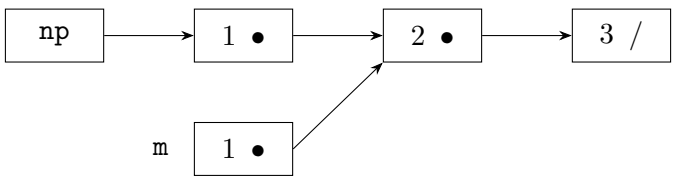
> *"Beggars can't be choosers."* — N. Naeem

Sometimes the free copy ctor does not do the right thing:

```
// node.h (include guard assumed)
struct Node {
    int data;
    Node *next;
    Node(int data, Node *next);
    Node(const Node &other);
};

// node.cc
Node::Node(int data, Node *next) : data{data}, next{next} {}
Node::Node(const Node &other) : data{other.data}, next{other.next} {}  // = free one

// main.cc
Node *np = new Node{1, new Node{2, new Node{3, nullptr}}};
Node m{*np};               // copy ctor; m is on the stack
Node *m2 = new Node{*np};  // copy ctor; *m2 is on the heap
```



This is not really a copy of the list: `m` shares nodes 2 and 3 with the original. This is called a **shallow copy**. For a pointer field we usually want to copy the object the pointer points to: a **deep copy**.

```
Node::Node(const Node &other)
    : data{other.data},
      next{other.next ? new Node{*other.next} : nullptr} {}
```

`new Node{*other.next}` is a recursive call to the copy ctor, which copies the rest of the list. The check is needed because `other.next` could be `nullptr`, and dereferencing it would crash.

13.3.1 When Is the Copy Constructor Called?

1. when an object is initialized from another object of the same type;
2. when an object is passed by value;
3. when an object is returned by value.

(2 and 3 may be optimized away or replaced by a move; see Lecture 11–12.)

Why the parameter is a reference

Because of (2), the copy ctor's parameter must be passed by reference: if it were `Node(Node other)`, passing the argument would need a copy, which calls the copy ctor, which needs a copy, ... infinite recursion. The language forces the reference.

13.4 Single-Parameter Constructors

```
Node::Node(int data) : data{data}, next{nullptr} {}

Node n{4};
Node n2 = 4;        // also compiles!
void foo(Node n);
foo(4);             // also compiles: 4 is converted to a Node
```

One-parameter ctors create *implicit* (automatic) conversions. This is exactly how `string s = "hello";` works: `std::string` has a one-parameter ctor taking a `const char*`. But for `Node`, silently turning an `int` into a Node is surprising and can hide bugs. To disable the conversion, use `explicit`:

```
struct Node {
    explicit Node(int data);
    ...
};
Node n{4};    // OK
Node n2 = 4;  // error
foo(4);       // error
foo(Node{4}); // OK
```

13.5 The Destructor

When an object is destroyed, a method called the **destructor** (dtor) runs. Objects are destroyed when

1. a stack-allocated object goes out of scope, or
2. a heap object is explicitly `delete`d.

Steps of object destruction

1. The dtor body runs.
2. Fields are destroyed: dtors are called for fields that are objects (in reverse declaration order).
3. The space is reclaimed.

A class has exactly one dtor, named `~ClassName()`, with no parameters and no return type. The free dtor has an empty body (and still does step 2). But that might not be what we want:

```
Node *np = new Node{1, new Node{2, new Node{3, nullptr}}};
delete np;    // calls the dtor for the node with value 1
```

`next` is a pointer, not an object, so step 2 does nothing for it: only the first node is freed, and nodes 2 and 3 are leaked. Let's write our own dtor:

```
Node::~Node() { delete next; }
```

This recursively deletes the rest of the list; the recursion ends at the last node because `delete nullptr` does nothing.

13.6 The Copy Assignment Operator

```
Student billy{60, 70, 80};
Student bobby{billy};   // copy ctor: bobby is being created
Student jane;
jane = billy;           // copy assignment: jane already exists
                        // same as jane.operator=(billy);
```

The free `operator=` does a field-for-field copy. Once a class manages heap memory, we need to write our own. Let's implement `operator=` for `Node`.

First, what should it return? Assignment associates to the right: `a = b = c = 0;` assigns `c = 0`, which produces `c`; then `b = c` produces `b`; then `a = b`. For objects,

```
n1 = n2 = n3;   // n1.operator=(n2.operator=(n3));
```

So `operator=` must return the object (by reference, to avoid a copy) to allow cascaded assignment.

```
Node &Node::operator=(const Node &other) {
    if (this == &other) return *this;   // self-assignment check: see below
    data = other.data;
    delete next;     // important: next may already own heap memory
    next = other.next ? new Node{*other.next} : nullptr;
    return *this;
}
```

The MIL cannot be used here: it is only available for constructors (the object already exists).

13.6.1 Why Check for Self-Assignment?

```
Node n{1, new Node{2, nullptr}};
n = n;       // n.operator=(n): unlikely to be written directly,
             // but *p = *q where p == q happens easily
```

Without the check, `other.next` is the same pointer as `next`. We `delete next`, which makes `other.next` a dangling pointer, and then `new Node{*other.next}` copies freed memory. Solution: check for self-assignment first.

Q. What happens if `new` fails (no memory)?

Then `next` has already been deleted, but has not been given a new value: the object is left holding a dangling pointer. Next lecture fixes this.

Summary

- The Big Five: copy ctor, copy assignment, dtor, move ctor, move assignment. The free versions copy/move field by field.
- With pointer fields, the free copy is shallow; write a deep copy.
- The copy ctor takes `const T&` and is used for initialization, pass-by-value and return-by-value.
- Mark one-parameter ctors `explicit` unless you want implicit conversion.
- Dtor: body, then fields, then space. Write one to free owned heap memory.
- `operator=` returns `*this` by reference, must free the old resources, and must handle self-assignment.

Chapter 14

Lecture 11: Copy-and-Swap, Rvalues and Move Semantics

February 8, 2018

14.1 Recap and the Failing-`new` Problem

```
Node &Node::operator=(const Node &other) {
    if (this == &other) return *this;
    data = other.data;
    delete next;
    next = other.next ? new Node{*other.next} : nullptr;
    return *this;
}
```

Q. What if `new` fails?

`new` throws an exception and the function is exited before `next` is assigned. If the caller catches the exception and tries to recover, the object's `next` is now a dangling pointer (and our dtor will delete it again later).

Solution. Don't delete the old data until the new data is safely in hand:

```
Node &Node::operator=(const Node &other) {
    if (this == &other) return *this;
    Node *tmp = next;
    next = other.next ? new Node{*other.next} : nullptr;  // if this throws,
    data = other.data;                                    // nothing has changed
    delete tmp;
    return *this;
}
```

(In fact this version is correct even without the self-assignment check, but the check saves work.)

14.2 The Copy-and-Swap Idiom

```
// node.h
struct Node {
    ...
    void swap(Node &other);
    Node &operator=(const Node &other);
};

// node.cc
#include <utility>
void Node::swap(Node &other) {
    using std::swap;
    swap(data, other.data);
    swap(next, other.next);
}

Node &Node::operator=(const Node &other) {
    Node tmp = other;   // deep copy via the copy ctor (may throw: fine)
    swap(tmp);          // I take tmp's fresh copy; tmp takes my old data
    return *this;       // tmp's dtor frees my old data
}
```

This relies on a correctly implemented copy ctor and dtor, and is safe for self-assignment and for a failing `new` automatically.

14.3 Member Functions vs. Standalone Functions

`operator=` must be implemented as a method; the left-hand side is represented by `*this`. Other operators can be either.

Example 14.1. Implement the Vec operators as methods.

```
// vec.h
struct Vec {
    int x, y;
    Vec operator+(const Vec &other) const;
    Vec operator*(int k) const;            // only v * k!
};

// vec.cc
#include "vec.h"
Vec Vec::operator+(const Vec &other) const {
    return {x + other.x, y + other.y};
}
Vec Vec::operator*(int k) const {
    return {x * k, y * k};
}
```

`k * v` cannot be a method of `Vec`, since the left operand is an `int`. It must remain a standalone function: `Vec operator*(int k, const Vec &v) { return v * k; }`.

What about I/O operators as methods?

```
struct Vec {
```

```
    ...
    std::ostream &operator<<(std::ostream &out) const {
        out << x << " " << y;
        return out;
    }
};

Vec v{1, 2};
v << cout;       // this compiles... and looks completely backwards
```

Rule

I/O operators should always be implemented as standalone functions, because the stream must be the left operand.

14.4 Tracing Copies

```
Node plusOne(Node n) {
    for (Node *p = &n; p; p = p->next) ++p->data;
    return n;
}

Node n{1, new Node{2, nullptr}};
Node n2{plusOne(n)};
```

- `n` is built with the basic ctor.
- Passing `n` by value to `plusOne`: copy ctor (deep copy of both nodes).
- Returning `n` by value from `plusOne`: copy into a temporary in `main`'s frame.
- Initializing `n2` from that temporary: copy ctor again.

Compiling with `g++ -std=c++14 -fno-elide-constructors node.cc` shows all the copy-ctor calls; without the flag the compiler removes some of them (copy elision, next lecture).

Look at the last step: `n2` is copy-constructed from a *temporary*, which is destroyed as soon as `n2` has been built. Deep-copying the whole list, only to throw the original away a moment later, is wasteful. When we are about to copy from a temporary, it makes sense to not copy but to take over (steal) its data.

14.5 Rvalues and Rvalue References

Q. How do we know a value is a temporary?

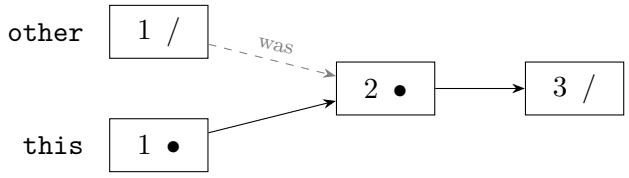
Definition 14.2. An **lvalue** is anything with an address (a named storage location). An **rvalue** is a value that is not an lvalue: temporaries such as `plusOne(n)`, `x + y`, or literals.

C++11 introduced **rvalue references**, written `T&&`, which bind to rvalues: references to temporary objects that are about to be destroyed. Overloading on `const T&` vs. `T&&` lets the compiler pick "copy" for lvalues and "steal" for rvalues.

14.6 The Move Constructor

The move ctor takes a single rvalue reference as its parameter:

```
Node::Node(Node &&other) : data{other.data}, next{other.next} {
    other.next = nullptr;   // other is about to be destroyed and its dtor
                            // will run: make sure we steal, not share
}
```



14.7 The Move Assignment Operator

```
Node m{10, new Node{20, nullptr}};
m = plusOne(n);           // the right side is an rvalue: move assignment

Node &Node::operator=(Node &&other) {
    using std::swap;
    swap(data, other.data);
    swap(next, other.next);   // or simply: swap(other);
    return *this;
}
```

Swapping gives `other` our old list; `other` is a temporary, so its dtor frees our old list for us. No allocation, so it cannot fail.

If you don't write the move operations, the copy versions are used for rvalues too (it is still correct, just slower).

14.8 Copy/Move Elision (Preview)

```
// c++/elision/vec.cc
Vec makeAVec() { return {0, 0}; }
Vec v = makeAVec();
```

Theoretically: one basic ctor (inside `makeAVec`) plus one copy/move ctor (to get the value into `v`). In practice only the basic ctor runs: the `Vec{0,0}` is created directly in the space for `v`. More next lecture.

Summary

- In `operator=`, allocate the new data before deleting the old, or use copy-and-swap.
- Operators whose left operand isn't your class (`k * v`, «, ») must be standalone functions.
- Rvalues are temporaries; `T&&` binds to them.

- Move ctor/assignment steal the resources of a dying object and leave it in a destructible state (e.g. `next = nullptr`).

Chapter 15

Lecture 12: Elision, Rule of Five, Const Methods and Encapsulation

February 13, 2018

15.1 Copy/Move Elision

(*Elision* means omission, disappearance.)

```
Vec makeVec() { return {0, 0}; }
...
Vec v = makeVec();
```

In theory: the basic ctor runs inside `makeVec`; then since we return by value and the returned value is an rvalue, the move ctor runs to build `v` (or the copy ctor if there is no move ctor).

What actually happens: only the basic ctor runs! The object is created directly in the space reserved for `v`.

Elision

Compilers are allowed (though, in C++14, not required) to eliminate copy and move constructions in these situations — even if the copy/move ctor has side effects (like printing). So never rely on the number of times a copy ctor runs.

To turn it off (e.g. to observe the calls): `g++ -fno-elide-constructors`.

Elision also applies to arguments:

```
void f(Vec v) { ... }
f(makeVec());    // the result is built directly in f's parameter
```

15.2 Rule of Five

Rule of Five

If you need to write any one of

- the copy constructor,
- the copy assignment operator,
- the destructor,
- the move constructor,
- the move assignment operator,

then you usually need to write all five.

The reason: needing a custom one of these almost always means the class *owns a resource* (heap memory, a file, ...), and every one of the five must then handle that resource. Most classes own nothing directly and need none of them.

15.3 Short Topic: Arrays of Objects

```
struct Vec {
    int x, y;
    Vec(int x, int y) : x{x}, y{y} {}
};

Vec vectors[3];              // stack array of 3 Vec objects
Vec *vp = new Vec[3];        // heap array of 3 Vec objects
```

Neither compiles: each element must be constructed, and there is no default ctor. How to fix this?

1. Provide a default ctor.
2. For a stack array, use array initialization syntax:

   ```
   Vec vectors[3] = {Vec{0, 0}, Vec{1, 3}, Vec{2, 4}};
   ```

3. Use an array of pointers to objects:

   ```
   Vec *vp[3];                    // stack array of 3 Vec pointers
   Vec **vp1 = new Vec*[3];       // heap array of 3 Vec pointers
   for (int i = 0; i < 3; ++i) vp1[i] = new Vec{i, i};
   ...
   for (int i = 0; i < 3; ++i) delete vp1[i];
   delete [] vp1;
   ```

15.4 Short Topic: Const Methods

```
struct Student {
    int assns, mt, final;
    float grade() { return 0.4 * assns + 0.2 * mt + 0.4 * final; }
};

const Student billy{80, 50, 75};   // can't change billy's fields
billy.grade();                     // error!
```

Q. Can I call `billy.grade()`?

No: `grade` did not promise not to change the fields, even though it doesn't actually change them. To make the promise, declare the method `const` (after the parameter list):

```
struct Student {
    int assns, mt, final;
    float grade() const { return 0.4 * assns + 0.2 * mt + 0.4 * final; }
};
```

Inside a const method, `this` has type `const Student*`, so the compiler rejects any modification of a field.

Rule

`const` objects can only call `const` methods. Make every method that doesn't modify the object `const`.

15.4.1 Mutable Fields

Suppose we want to count how many times `grade` is called:

```
struct Student {
    int assns, mt, final;
    int numCalls = 0;
    float grade() const {
        ++numCalls;     // error: grade is const
        return 0.4 * assns + 0.2 * mt + 0.4 * final;
    }
};
```

This won't compile. But the call counter isn't really part of the student's logical state, so a const method should be allowed to change it. Mark the field `mutable`:

```
    mutable int numCalls = 0;
```

A `mutable` field can be modified in a const object or by a const method. (Distinguish *physical* constness — no bits change — from *logical* constness — the object's observable state doesn't change.)

15.5 Invariants and Encapsulation

```
struct Node {
    int data;
    Node *next;
    Node(int data, Node *next) : data{data}, next{next} {}
    ~Node() { delete next; }
};

Node n1{1, new Node{2, nullptr}};   // fine
Node n2{3, nullptr};                // fine
Node n3{4, &n2};                    // disaster
```

When `n3` goes out of scope, its dtor runs `delete &n2`, deleting a stack address: the program is likely to crash.

Definition 15.1. An **invariant** is a statement/assumption about a class that must be true for the class to function correctly.

- Node invariant: `next` is either `nullptr` or points to a heap-allocated Node (that this Node owns).
- Stack invariant: the last item pushed is the first popped. Easy to break if clients can reach the underlying array and, say, sort it.

If client code has unrestricted access to the implementation, it can break invariants, resulting in crashes and undefined behaviour.

Definition 15.2. Encapsulation: classes should be treated as black boxes (capsules). Client code gets limited access through public methods (the *interface*).

```
struct Vec {
    Vec(int x, int y);        // public: struct members are public by default
  private:
    int x, y;
  public:
    Vec operator+(const Vec &v) const { return {x + v.x, y + v.y}; }
};

int main() {
    Vec v{1, 2};              // OK: Vec(int, int) is public
    Vec v1 = v + v;           // OK: operator+ is public; free Big 5 are public
    cout << v.x << endl;      // won't compile: x is private
}
```

Advice

Fields should be private. Only selected methods should be public (the interface).

15.5.1 The `class` Keyword

```
class Vec {
    int x, y;                 // private by default
  public:
    Vec(int x, int y);
    Vec operator+(const Vec &v) const;
};
```

The *only* difference between `struct` and `class` is the default visibility: public for `struct`, private for `class`.

15.6 Encapsulating Node

To guarantee the Node invariant, create a wrapper class `List` that has exclusive access to the Node objects:

```
// list.h
class List {
    struct Node;                // private nested class: declared only
    Node *theList = nullptr;
  public:
    void addToFront(int n);
    int ith(int i) const;
    ~List();
};

// list.cc
#include "list.h"
struct List::Node {             // full definition hidden in the .cc file
    int data;
    Node *next;
    Node(int data, Node *next) : data{data}, next{next} {}
    ~Node() { delete next; }
};

List::~List() { delete theList; }

void List::addToFront(int n) {
    theList = new Node{n, theList};
}

// requires: 0 <= i < length of the list
int List::ith(int i) const {
    Node *curr = theList;
    for (int j = 0; j < i && curr; ++j, curr = curr->next);
    return curr->data;
}
```

No one outside `List` knows the Node structure (it is defined only in the `.cc` file), so only `List` can create Nodes, and it always puts them on the heap. We now have a guarantee that the invariant is maintained. (We should also write or disable the copy/move operations of `List`: Rule of Five.)

The cost. The client can no longer walk the list. Printing it requires `ith(0)`, `ith(1)`, ..., each of which walks from the front, so traversing the list costs $1 + 2 + \cdots + n \in \Theta(n^2)$ instead of $\Theta(n)$. Next lecture: the Iterator pattern fixes this.

Summary

- Compilers may elide copies/moves; don't rely on their side effects.
- Rule of Five: write all five or none.
- Arrays of objects need a default ctor, an initializer, or pointers.
- `const` methods can be called on const objects; `mutable` fields are exempt from constness.
- Invariants must be protected by encapsulation: private fields, public interface; `class` defaults to private.
- Hiding Node inside List secures the invariant but makes traversal $\Theta(n^2)$.

Chapter 16

Lecture 13: Iterator Pattern, Friends and Static Members

February 15, 2018

The encapsulated `List` has $\Theta(n^2)$ traversal time. We want to bring it back down to $\Theta(n)$ without exposing the Node class.

16.1 Design Patterns

Definition 16.1. A **design pattern** is a commonly occurring, well-tested solution to a recurring design problem: "if you have *this* situation, then *this* technique may help".

16.2 The Iterator Pattern

We need to keep track of how much of the list we have already traversed, without exposing Nodes. Create an `Iterator` class, which is given access to the nodes. It

- keeps track of where we are in the traversal, and
- acts like a pointer into the list.

Inspiration — walking an array with a pointer:

```
for (int *p = array; p != array + arraySize; ++p) {
    ... *p ...
}
```

So the iterator should support `!=`, unary prefix `++`, and unary `*`. We also need a way to create an iterator at the start, and a way to tell that we have reached the end.

```
class List {
    struct Node;
    Node *theList = nullptr;
  public:
    class Iterator {
        Node *curr;
      public:
        explicit Iterator(Node *curr) : curr{curr} {}
```

```
        int &operator*() const { return curr->data; }
        Iterator &operator++() {          // prefix ++
            curr = curr->next;
            return *this;
        }
        bool operator!=(const Iterator &other) const {
            return curr != other.curr;
        }
    };  // end of Iterator

    Iterator begin() { return Iterator{theList}; }
    Iterator end() { return Iterator{nullptr}; }
    void addToFront(int n);
    ...
};  // end of List
```

(The methods of `Iterator` touch `curr->data`, so in practice they are defined in `list.cc`, after `List::Node` is defined.)

```
List l;
l.addToFront(1);
l.addToFront(2);
l.addToFront(3);
for (List::Iterator it = l.begin(); it != l.end(); ++it) {
    cout << *it << endl;     // prints 3, 2, 1 -- in Theta(n) total
}
```

16.2.1 `auto`

As of C++11, `auto x = y;` declares `x` with the same type as `y` (automatic type deduction). So the loop header can be `for (auto it = l.begin(); it != l.end(); ++it)`.

16.2.2 Range-Based For Loops

C++11 has built-in support for the Iterator pattern:

```
for (auto n : l) {        // n is a copy of each element (type of *it)
    cout << n << endl;
}
for (auto &n : l) {       // n is a reference: we can modify the list
    n += 1;
}
```

This works for any class `C` such that

1. `C` has methods `begin()` and `end()` that return objects of some class `D`, and
2. `D` has `operator*`, prefix `operator++` and `operator!=`.

The range-based for loop is just syntactic sugar for the explicit loop above; it makes the programmer's job easier.

16.3 Friends

We want client code to get iterators only through `begin()` and `end()` (otherwise a client could do `List::Iterator{nullptr}` or worse). So the Iterator ctor should be private. But then `List` itself could not call it either.

The `Iterator` class can grant its private members to `List` by declaring `List` a **friend**:

```
class List {
    struct Node;
    Node *theList = nullptr;
  public:
    class Iterator {
        Node *curr;
        explicit Iterator(Node *curr) : curr{curr} {}  // now private
      public:
        int &operator*() const;     // by reference: clients can read/write
        Iterator &operator++();     // returns Iterator&: ++ is an update
        bool operator!=(const Iterator &other) const;
        friend class List;          // List may call the private ctor
    };
    Iterator begin() { return Iterator{theList}; }
    Iterator end() { return Iterator{nullptr}; }
    ...
};
```

Friendship

Friendship breaks encapsulation: a friend can touch all private members. Have as few friends as possible.

16.4 Accessors and Mutators

Advice

Keep fields private.

To provide controlled read/write access, use **accessors** (getters) and **mutators** (setters). A mutator can enforce invariants.

```
class Vec {
    int x, y;
  public:
    int getX() const { return x; }
    int getY() const { return y; }
    void setX(int v) { x = v; }
    void setY(int v) { y = v; }
};
```

16.4.1 I/O Operators and Friends

I/O operators are standalone functions, so they can't see private fields. If there are no suitable accessors, we can make the function a friend:

```
class Vec {
    int x, y;
  public:
    ...
    friend std::ostream &operator<<(std::ostream &out, const Vec &v);
};

std::ostream &operator<<(std::ostream &out, const Vec &v) {
    out << v.x << " " << v.y;
    return out;
}
```

Note that this does *not* make `operator«` a method; it is still a standalone function, just one with access to Vec's privates.

16.5 Static Fields and Static Methods

Definition 16.2. A **static field** is associated with the class itself, not with individual objects. Think of it as a per-class global variable: there is exactly one copy, shared by all objects.

```
// student.h
struct Student {
    static int objCount;
    int assns, mt, final;
    Student(int assns, int mt, int final)
        : assns{assns}, mt{mt}, final{final} {
        ++objCount;
    }
};
```

C++ rule

A static field must be *defined* outside the class, in exactly one `.cc` file:

```
// student.cc
int Student::objCount = 0;
```

16.5.1 Static Member Functions

- A static member function is associated with the class, not an object.
- It does not need an object to be called on, and so it has no `this` pointer.
- Consequently it can only access static fields and call other static member functions.

```
struct Student {
    static int objCount;
    static void printObjCount() {
        cout << objCount << endl;
    }
```

```
    ...
};

Student billy{60, 70, 80}, bobby{70, 80, 90};
Student::printObjCount();   // outputs 2
```

Summary

- A design pattern is a reusable solution to a common design problem.
- Iterator: a nested class that remembers a position and offers `*`, `++`, `!=`; `begin()`/`end()` create them; range-based for uses exactly this protocol.
- `friend` grants access to private members; use sparingly.
- Keep fields private; use accessors/mutators.
- Static fields/methods belong to the class; define static fields in a `.cc` file; static methods have no `this`.

Chapter 17

Lecture 14: System Modelling, UML and Inheritance

February 27, 2018

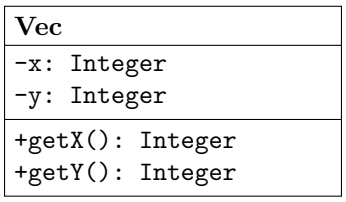
17.1 System Modelling

It is a good idea to sketch out a design before you code. Designing a program means

- identifying the key entities (classes), and
- identifying the relationships between classes — this is the most important part.

17.2 UML

UML (Unified Modelling Language) is a standard notation for drawing class diagrams. A class is drawn as a box with three sections: name, fields, methods. Visibility is marked with `+` (public), `-` (private), `#` (protected).



Fields and methods may be omitted when they are not relevant to the diagram.

17.3 Relationship 1: Composition (owns-a)

```
class Vec {
    int x, y;
  public:
    Vec(int x, int y) : x{x}, y{y} {}
};
```

```
class Basis {
    Vec v1, v2;
};

Basis b;   // won't compile: Vec has no default ctor
```

To make it compile, either

1. give `Vec` a default ctor, or
2. have `Basis`'s ctor use the MIL to call a non-default Vec ctor:

```
class Basis {
    Vec v1, v2;
  public:
    Basis() : v1{1, 0}, v2{0, 1} {}
};
Basis b;   // compiles
```

Embedding one object inside another is called **composition**: a Basis **owns-a** Vec (two, in fact). If A owns-a B, then typically

1. B has no identity outside A;
2. if A is destroyed, B is destroyed;
3. if A is copied, B is copied (deep copy).

Example: a car owns its wheels. In UML, composition is drawn with a *filled* diamond at the owner's end; multiplicities go at the other end.



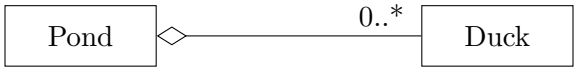
17.4 Relationship 2: Aggregation (has-a)

Example: parts in a catalogue, ducks in a pond. A catalogue **has-a** collection of parts. If A has-a B, then typically

1. B exists independently of A;
2. destroying A does not destroy B;
3. copying A does not copy B (shallow copy: the copies share the B's).

```
class Catalogue {
    Part *parts[100];   // pointers to parts that live elsewhere
    ...                 // ~Catalogue() does NOT delete them
};
```

In UML, aggregation is drawn with a *hollow* diamond.



Whether a pointer field means ownership or not is not visible in the code; it is a design decision that must be documented (and reflected in the Big Five).

17.5 Relationship 3: Inheritance (is-a)

Suppose we have classes `Book` (title, author, numPages), `Text` (additionally a topic), and `Comic` (additionally a hero). As three unrelated classes they capture no relationship.

Q. How do we create an array that can hold all of these?

In C we would use an array of `void*`, or an array of a `union` type:

```
union BookTypes {
    Book *b;
    Text *t;
    Comic *c;
};
BookTypes myBooks[100];
```

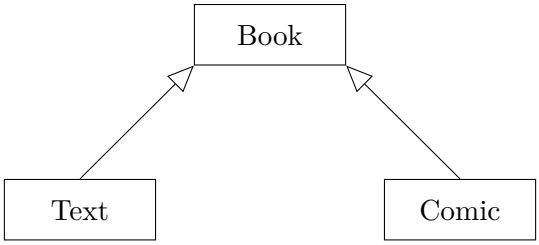
Neither is type-safe: nothing records which kind of book each element is.

Observe: a Text *is a* Book (with a topic field), and a Comic *is a* Book (with a hero field). Model this with inheritance:

```
class Book {                        // base class / superclass
    string title, author;
    int numPages;
  public:
    Book(const string &t, const string &a, int n)
        : title{t}, author{a}, numPages{n} {}
};

class Text : public Book {          // derived class / subclass
    string topic;
  public:
    Text(const string &t, const string &a, int n, const string &topic);
};

class Comic : public Book {
    string hero;
  public:
    Comic(const string &t, const string &a, int n, const string &hero);
};
```



A derived class inherits all the members (fields and methods) of its base class. Any method you can call on a Book, you can call on a Text or Comic.

> We've seen this already: anything you can do with an `istream` you can do with an `ifstream` or `istringstream`, because they inherit from `istream`.

17.5.1 Inheriting Private Members

`Text` has inherited `title`, `author` and `numPages`, but they are private in `Book`:

```
Text t{...};
t.author = "Bob";    // error: author is private
```

Private really does mean private to the class — even subclasses can't access it. So this constructor does not compile:

```
Text::Text(const string &t, const string &a, int n, const string &topic)
    : title{t}, author{a}, numPages{n}, topic{topic} {}   // error
```

for two reasons:

1. `title`, `author`, `numPages` are private in Book;
2. the MIL can only name fields declared in *this* class.

Object construction, with inheritance

1. Space is allocated.
2. The superclass part is constructed.
3. Fields are initialized (MIL or default ctors).
4. The ctor body runs.

In step 2 the Book part is default-constructed unless told otherwise — but Book has no default ctor. Options: (1) give Book a default ctor; (2) call a non-default Book ctor in the MIL:

```
Text::Text(const string &t, const string &a, int n, const string &topic)
    : Book{t, a, n}, topic{topic} {}
```

Option 2 is the right one: the superclass ctor might do work (like enforcing invariants) that subclasses shouldn't have to repeat.

17.6 Protected Visibility

A `protected` member is accessible by the class and all of its subclasses (but not by clients).

```
class Book {
  protected:
    string title, author;
    int numPages;
  public:
    ...
};
class Text : public Book {
  public:
    void addAuthor(const string &a) { author += a; }   // OK: protected
};
Text t{...};
t.author = "Bob";    // still an error for clients
```

But protected fields break encapsulation: anyone can write a subclass and thereby get access, and a subclass may break the base class's invariants.

Advice

Keep fields private; give subclasses protected accessors/mutators instead.

```
class Book {
    string title, author;
    int numPages;
  protected:
    void addAuthor(const string &a) { author += a; }   // controlled access
  public:
    ...
};
```

Summary

- Design first: classes and relationships, drawn in UML.
- Composition (owns-a, filled diamond): parts die and copy with the whole.
- Aggregation (has-a, hollow diamond): parts live independently.
- Inheritance (is-a, open triangle): subclasses inherit all members, but can't touch private ones; construct the base part through the MIL.
- `protected` gives subclasses access; prefer private fields with protected accessors.

Chapter 18

Lecture 15: Method Overriding, Virtual Methods and Polymorphism

March 1, 2018

18.1 Overriding

We want to decide whether a book is heavy: a Book is heavy if it has more than 200 pages, a Text if more than 500, a Comic if more than 30.

```
class Book {
    ...
    int numPages;
  public:
    int getNumPages() const { return numPages; }
    bool isHeavy() const { return numPages > 200; }
};

class Comic : public Book {
    ...
  public:
    bool isHeavy() const { return getNumPages() > 30; }   // overriding
};
// Text is similar, with 500.

Book b{"title", "author", 50};
cout << b.isHeavy();     // Book::isHeavy: false

Comic c{"title", "author", 40, "hero"};
cout << c.isHeavy();     // Comic::isHeavy: true

Book b1 = Comic{"title", "author", 40, "hero"};   // compiles!
cout << b1.isHeavy();    // Book::isHeavy runs: false
```

Definition 18.1. Method overriding: a subclass replacing the implementation of an inherited method. (Not to be confused with *overloading*: same name, different parameter lists.)

18.2 Object Slicing

In `Book b1 = Comic{...};`, `b1` is a Book: there is only enough space for a Book's fields. The Comic is copied using Book's copy ctor, and the `hero` field is chopped off. This is called **object slicing** (another professor calls it *object coercion*). So `b1` really is a Book, and `Book::isHeavy` runs.

Slicing does not occur if a subclass object is pointed to by a superclass pointer (or reference):

```
Comic c{"title", "author", 40, "hero"};
Comic *cp{&c};
cout << cp->isHeavy();   // Comic::isHeavy: true
Book *bp{&c};            // fine: a Comic is-a Book
cout << bp->isHeavy();   // Book::isHeavy runs: false!
```

The Comic object only acts like a Comic when a Comic pointer points to it. For non-virtual methods, the compiler looks at the *declared* type of the pointer and decides at compile time which method to run.

But most of the time this is not what we want:

```
Book *myCollection[100];
...
for (int i = 0; i < 100; ++i) {
    cout << myCollection[i]->isHeavy() << endl;   // always Book::isHeavy
}
```

We want the "most specific" `isHeavy` to run for each book.

18.3 Virtual Methods

```
class Book {
    ...
  public:
    virtual bool isHeavy() const { return numPages > 200; }
};

class Comic : public Book {        // same for Text
    ...
  public:
    bool isHeavy() const override { return getNumPages() > 30; }
};
```

The keyword `override` is optional, but recommended: if you make a mistake in the signature (e.g. forget the `const`), the method would silently be a new, unrelated method; with `override` it won't compile.

```
Comic c{"title", "author", 40, "hero"};
Comic *cp{&c};
Book *bp{&c};
Book &rb{c};
cout << cp->isHeavy();   // Comic::isHeavy
cout << bp->isHeavy();   // Comic::isHeavy
cout << rb.isHeavy();    // Comic::isHeavy
```

Definition 18.2. For a **virtual** method, the method to call is chosen while the program is running, based on the *run-time type* of the object, not the declared type of the pointer/reference. This is **dynamic dispatch**.

- Dynamic dispatch has a (small) run-time cost (see the chapter on vtables).
- C++'s default is **static dispatch** (non-virtual): you don't pay for what you don't use.

The run-time type is genuinely not known at compile time:

```
Book *bp;
if (userSaysComic) bp = &c; else bp = &t;
cout << bp->isHeavy();   // decided at run time

Book *myBooks[100];      // store Books, Comics and Texts
for (int i = 0; i < 100; ++i) {
    cout << myBooks[i]->isHeavy() << endl;   // the right one for each
}
```

`myBooks` is called a *polymorphic array*.

Definition 18.3. Polymorphism ("many forms"): the ability of one variable/container to accommodate objects of multiple types.

We have used it all along: in

```
ostream &operator<<(ostream &out, const Vec &v);
```

`out` is a polymorphic parameter: we can pass `cout`, an `ofstream`, or an `ostringstream`.

```
Vec v{1, 2};
cout << v;
ofstream file{"vec.txt"};
file << v;               // same operator<<
```

18.4 Destructors Revisited

Object destruction is the reverse of object construction. When a subclass object is destroyed:

1. the subclass dtor body runs;
2. fields of the subclass that are objects are destroyed;
3. the superclass part is destroyed (the superclass dtor is called automatically);
4. the space is released.

```
// c++/inheritance/example5
class X {
    int *x;
  public:
    X(int n) : x{new int[n]} {}
    ~X() { delete [] x; }
};
```

```
class Y : public X {
    int *y;
  public:
    Y(int n, int m) : X{n}, y{new int[m]} {}
    ~Y() { delete [] y; }      // X's dtor is called automatically after
};

Y myY{10, 20};          // fine: ~Y then ~X at end of scope
X *myX = new Y{10, 20}; // fine: a Y is-a X
delete myX;             // calls ~X() only!
```

The dtor is non-virtual, so `delete myX` uses the declared type: only `~X` runs, and `y`'s array is leaked. Fix: make the base class dtor virtual (which makes all subclass dtors virtual too):

```
class X {
    ...
  public:
    virtual ~X() { delete [] x; }
};
```

Rule

Always make the base class dtor virtual, even if its body is empty, if the class might ever have subclasses.

If a class should never be subclassed, declare it `final`:

```
class Y final : public X { ... };   // Y can't be subclassed
class Z : public Y { ... };         // won't compile
```

{C++: gotta love it.} `final` and `override` are only keywords in these particular positions; you can still name a variable `override`.

Summary

- Overriding replaces an inherited method's implementation.
- Assigning a subclass object to a superclass *object* slices it; pointers and references don't slice.
- Non-virtual methods use static dispatch (declared type); virtual methods use dynamic dispatch (run-time type). Mark overrides with `override`.
- Polymorphism: one pointer/reference type, many object types.
- Base class dtors must be virtual; use `final` to forbid subclassing.

Chapter 19

Lecture 16: Abstract Classes, Templates, Vectors and Exceptions

March 6, 2018

19.1 Pure Virtual Methods and Abstract Classes

```
class Student {
    ...
  public:
    virtual int fees() const = 0;    // pure virtual method
    virtual ~Student() {}
};

class Regular : public Student {
  public:
    int fees() const override { ... }  // regular fees
};

class CoOp : public Student {
  public:
    int fees() const override { ... }  // co-op fees
};
```

What should `Student::fees` do? There's no sensible answer: every student is either regular or co-op. So we don't implement it.

Definition 19.1. A **pure virtual method** (declared with `= 0`) is a method that need not have an implementation in its class. A class with at least one pure virtual method is an **abstract class**, and cannot be instantiated. A class that is not abstract is **concrete**.

```
Student s;                    // won't compile: Student is abstract
Student *sp = new CoOp{...};  // fine
```

A subclass inherits all members, including pure virtual methods; so a subclass must implement *all* inherited pure virtual methods to be concrete, otherwise it is abstract too.

Uses of abstract classes:

- organize subclasses;
- hold the fields and methods common to all subclasses;
- serve as the type of pointers/references, so polymorphism works: a `Student*` points to either a Regular or a CoOp student (never to a plain Student, since none can exist).

UML

In UML, virtual and pure virtual methods are written in *italics*, and so are the names of abstract classes. Static members are underlined. (Tools: StarUML, OmniGraffle, ...)

19.2 Templates

```
class Stack {
    int size, cap;
    int *contents;
  public:
    Stack();
    void push(int x);
    void pop();
    int top() const;
    ~Stack();
};
```

This works only for `int`s. A stack of `string`s or of `Expression*`s would be the same code with a different type.

Definition 19.2. A **template class** is a class parameterized on one or more types.

```
template <typename T> class Stack {
    int size, cap;
    T *contents;
  public:
    Stack();
    void push(T x);
    void pop();
    T top() const;
    ~Stack();
};

Stack<int> sInts;           // template instantiation with T = int
sInts.push(1);
int x = sInts.top();

Stack<Expression*> sExpr;
sExpr.push(new BinExp{...});
```

The compiler takes the template and generates (stamps out) a separate copy of the class for each type it is used with, substituting the type for `T`. Because of this, the full template implementation must be visible where it is used: templates are written entirely in the header file.

19.2.1 A Template List with an Iterator

```
template <typename T> class List {
    struct Node {
        T data;
        Node *next;
        Node(const T &data, Node *next) : data{data}, next{next} {}
        ~Node() { delete next; }
    };
    Node *theList = nullptr;
  public:
    class Iterator {
        Node *curr;
        explicit Iterator(Node *curr) : curr{curr} {}
      public:
        T &operator*() const { return curr->data; }
        Iterator &operator++() { curr = curr->next; return *this; }
        bool operator!=(const Iterator &other) const {
            return curr != other.curr;
        }
        friend class List<T>;
    };
    Iterator begin() { return Iterator{theList}; }
    Iterator end() { return Iterator{nullptr}; }
    void addToFront(const T &n) { theList = new Node{n, theList}; }
    T &ith(int i);
    ~List() { delete theList; }
};
```

```
List<int> l1;
l1.addToFront(1);

List<List<int>> l2;     // a list of lists of ints
l2.addToFront(l1);      // (needs a correct copy ctor for List!)

for (auto &m : l2) {
    for (auto n : m) cout << n << endl;
}
```

19.3 The STL: Dynamic-Length Arrays

The **Standard Template Library** provides many template classes. The most useful is `std::vector`:

- guaranteed to be implemented as an array (elements are contiguous);
- automatically resizes as needed.

```
#include <vector>
using namespace std;

int main() {
```

```
    vector<int> v{4, 5};   // [4, 5]; v is on the stack, its array on the heap
    v.emplace_back(6);     // [4, 5, 6]
    v.push_back(7);        // [4, 5, 6, 7]
    vector<int> w(4, 5);   // round brackets: 4 elements, each 5: [5, 5, 5, 5]
}
```

Note the difference between `{4, 5}` (the elements) and `(4, 5)` (count and value).

Ways to loop over a vector:

```
for (size_t i = 0; i < v.size(); ++i) cout << v[i] << endl;  // operator[]

for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
    cout << *it << endl;

for (auto n : v) cout << n << endl;

for (vector<int>::reverse_iterator it = v.rbegin(); it != v.rend(); ++it)
    cout << *it << endl;                    // backwards
for (auto it = v.rbegin(); it != v.rend(); ++it) ...
```

Highly recommended: read the reference for `vector`.

```
v.pop_back();             // remove the last element
v.erase(v.begin());       // remove the first element: O(n)
v.erase(v.end() - 1);     // remove the last element
```

The STL makes heavy use of iterators. `vector::erase` takes an iterator, and returns an iterator to the element after the one just erased (iterators to erased or later elements are invalidated).

Assignment 4

A4 prohibits heap arrays (`new Abc[n]`): use `vector` instead. A vector manages its own memory, so classes using it need no Big Five.

19.3.1 Checked and Unchecked Access

`v[i]` accesses the ith element *unchecked*: the program does not check whether `i` is in bounds, and an out-of-bounds access is undefined behaviour. `v.at(i)` is *checked* access (Java does checked access by default).

Q. What happens if the argument to `at` is out of range?

A. An exception is thrown (raised).

19.4 C++ Exceptions

```
// client code
vector<int> v;
v.at(5);     // vector::at detects the error -- but it is the client
             // who knows how to deal with it
```

Q. How should `vector::at` report the error to the client?
In C:

- return an error value (e.g. -1) — but every `int` might be a legitimate element;
- set a global error variable, e.g. `errno`.

Both are bad:

- awkward to code with `if` statements after every call;
- programmers are lazy, so errors go undetected.

"You are to stop being lazy right off the bat."

In C++, when an error condition occurs, the function **throws an exception**. Control immediately leaves the function and goes back up the call chain looking for a *handler*. If nobody handles it, the program terminates. So the error can't be silently ignored.

```
#include <stdexcept>
vector<int> v;
try {
    cout << v.at(10000) << endl;   // throws std::out_of_range
    cout << "never printed" << endl;
} catch (out_of_range &r) {
    cerr << "Range error: " << r.what() << endl;
}
```

`what()` returns a string describing the error. The details follow next lecture.

Summary

- Pure virtual (`= 0`) methods make a class abstract; subclasses must implement them all to be concrete.
- Templates parameterize classes on types; they live in headers.
- `std::vector`: a self-resizing array; iterate with indices, iterators, range-for or reverse iterators; `erase` takes iterators.
- `v[i]` is unchecked; `v.at(i)` throws `out_of_range`.
- Exceptions replace error codes: they can't be ignored, and are caught with `try`/`catch`.

Chapter 20

Lecture 17: Exceptions and Design Patterns

March 8, 2018

20.1 Exceptions (continued)

Last time: in C++, when an error condition occurs, an exception is thrown. By default, an exception that is not handled causes the program to terminate.

To handle exceptions we use a `try { ... }` block followed by `catch (...) { ... }` blocks (see `rangeErrorCaught.cc`):

```
#include <iostream>
#include <stdexcept>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    v.emplace_back(2);
    try {
        cout << v.at(1) << endl;    // out of range
    } catch (out_of_range &r) {
        cerr << "Range error: " << r.what() << endl;
    }
    cout << "Program continues" << endl;
}
```

20.1.1 Stack Unwinding

The thrower and the handler don't have to be in the same function (`callChain.cc`):

```
void f() {
    cout << "Start f" << endl;
    throw out_of_range{"f"};        // the argument becomes what()
    cout << "Finish f" << endl;     // never printed
}
void g() { cout << "Start g" << endl; f(); cout << "Finish g" << endl; }
```

```
void h() { cout << "Start h" << endl; g(); cout << "Finish h" << endl; }

int main() {
    cout << "Start main" << endl;
    try {
        h();
    } catch (out_of_range &) {
        cerr << "Range error" << endl;
    }
    cout << "Finish main" << endl;
}
// Output: Start main, Start h, Start g, Start f, Range error, Finish main
```

Definition 20.1. Stack unwinding: when an exception is thrown, the call stack is repeatedly popped until a matching `catch` block is found. Destructors of stack-allocated objects in the popped frames run as their frames are popped.

20.1.2 Recovery in Stages

A handler can do part of the recovery and then pass the problem on:

```
try {
    ...
} catch (SomeException &e) {
    // do part of the recovery, then choose:
    throw SomeOtherException{...};  // 1. throw a completely different exception
    throw e;                        // 2. throw the exception we caught
    throw;                          // 3. re-throw the original: recommended
}
```

In option 2, the exception we caught might really have been a *subclass* of `SomeException`; `throw e` throws a copy made from the *static* type of `e`, i.e. a sliced object. In option 3, the original exception object is re-thrown as is; no slicing.

20.1.3 What Can Be Thrown

- All exceptions in the C++ standard library inherit from the class `std::exception` (e.g. `out_of_range`, `invalid_argument`, `bad_alloc`, `length_error`).
- User-defined exceptions do not have to inherit from `exception`.
- In C++ you can throw anything: `int`s, strings, whatever.

Q. How do we write a catch block that catches all kinds of exceptions?

```
try {
    ...
} catch (...) {     // literally three dots: catches everything
    ...
}
```

20.1.4 Exceptions for Control Flow?

Since exceptions jump out of arbitrarily deep calls, you could use them for normal control flow, e.g. returning a result from deep recursion by throwing it.

> *"You can do some cool shit with exceptions, but don't use it."* — Nomair Naeem

Exceptions are slow to process: compare `recfib.cc` (ordinary recursive Fibonacci) with `exfib.cc` (the same, but each call "returns" by throwing): the throwing version is dramatically slower. Use exceptions for actual errors.

20.1.5 Good Practice

Use the existing exception classes or create your own:

```
class BadInput {};

int n;
try {
    if (!(cin >> n)) throw BadInput{};
} catch (BadInput &) {              // naming the parameter (&b) is also legal
    cout << "Not an int, using 0" << endl;
    n = 0;
}
```

Advice

Always catch exceptions by reference:

- no slicing occurs (subclass information, and virtual `what()`, are preserved);
- it is more efficient, since no copy is made.

Exceptions to know: `vector::at` throws `std::out_of_range`; `new` throws `std::bad_alloc` when there is no more heap memory.

20.1.6 Exceptions and Destructors

- By default, if a destructor throws an exception, the program terminates (`std::terminate` is called), because dtors are implicitly `noexcept`.
- It is possible to write a destructor that may throw: `~Node() noexcept(false) { ... }`. This is a bad idea.

Why: suppose the stack is unwinding because of one live exception. As frames are popped, destructors run. If one of those destructors throws, there are now two live exceptions, and C++ has no way to handle both: `std::terminate` is called.

20.2 Design Patterns (continued)

The guiding principle behind design patterns: **program to an interface, not an implementation**.

- Use abstract base classes to provide the interface.
- Work through pointers/references to the abstract class and call its virtual methods.
- The actual objects can then be swapped in or out as needed: we get abstraction over a variety of behaviours.

20.3 Iterator Pattern Revisited

Our iterator implemented `*`, `++` and `!=`. Make that an interface:

```
class AbsIter {
  public:
    virtual int &operator*() const = 0;
    virtual AbsIter &operator++() = 0;
    virtual bool operator!=(const AbsIter &other) const = 0;
    virtual ~AbsIter() {}
};

class List {
    struct Node;
    ...
  public:
    class Iterator : public AbsIter {
        // override the three pure virtual methods
    };
};

class Set {
    ...
  public:
    class Iterator : public AbsIter {
        ...
    };
};
```

Since `List::Iterator` and `Set::Iterator` both implement the `AbsIter` interface, we can write generic traversals that work on either data structure:

```
template <typename Fn>              // Fn: the type of a function parameter
void foreach(AbsIter &start, const AbsIter &finish, Fn f) {
    while (start != finish) {
        f(*start);
        ++start;
    }
}

void addFive(int &x) { x += 5; }

List l;
...
List::Iterator b = l.begin();
foreach(b, l.end(), addFive);       // adds 5 to every element of l
```

20.4 The Observer Pattern

Used for publish/subscribe systems:

- the entity publishing data is the **subject** (publisher);
- the entities subscribing to the data are **observers** (subscribers).

The protocol:

1. new data becomes available in the subject;
2. the subject notifies all attached observers (`notifyObservers()` calls each observer's `notify()`);
3. each observer requests whatever further information it needs from the subject, e.g. by calling `getState()`.

Example: a spreadsheet's cells (subjects) and the charts and formulas that depend on them (observers). Details and code next lecture.

Summary

- `try`/`catch`; stack unwinding pops frames (running dtors) until a handler is found.
- Re-throw with plain `throw;` to avoid slicing; catch by reference; `catch (...)` catches everything.
- Exceptions are for errors, not control flow (they are slow).
- Dtors should never throw: two simultaneous exceptions terminate the program.
- Program to interfaces; an abstract iterator enables generic algorithms.
- Observer: subjects notify attached observers of changes.

Chapter 21

Lecture 18: Observer and Decorator Patterns, Big Five Revisited

March 13, 2018

21.1 The Observer Pattern

Last time: the subject publishes, observers subscribe.

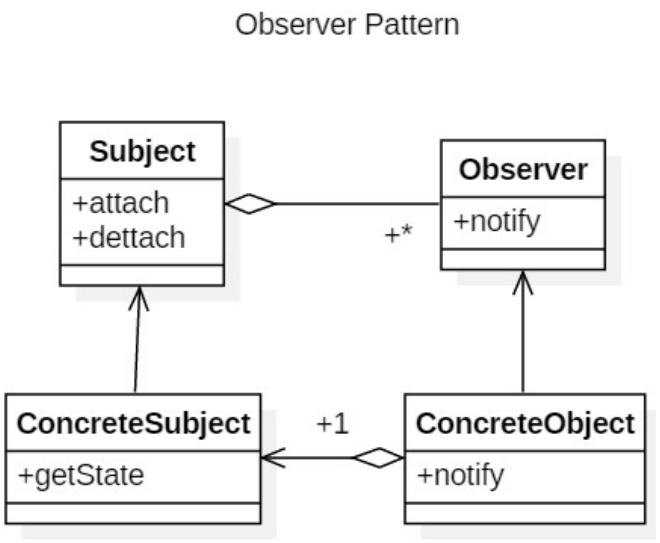


Figure 21.1: Observer pattern (UML). A Subject aggregates any number of Observers; each ConcreteObserver knows its ConcreteSubject so it can call `getState()`.

```
class Observer {
  public:
    virtual void notify() = 0;
    virtual ~Observer() {}
};
```

```
class Subject {
    std::vector<Observer*> observers;
  public:
    void attach(Observer *o) { observers.emplace_back(o); }
    void detach(Observer *o) {
        for (auto it = observers.begin(); it != observers.end(); ++it) {
            if (*it == o) { observers.erase(it); break; }
        }
    }
    void notifyObservers() {
        for (auto &o : observers) o->notify();
    }
    virtual ~Subject() = 0;
};
```

We want `Subject` to be abstract, but it has no obvious pure virtual method (every subject attaches/detaches/notifies the same way). Trick: make the *destructor* pure virtual.

Problem. The dtor of a subclass of `Subject` will call the base class's dtor — which has no implementation, so the program won't link.

Solution. Implement `Subject`'s dtor anyway:

```
// subject.cc
Subject::~Subject() {}
```

A pure virtual method *may* have an implementation. "Pure virtual" means a subclass must override it to be concrete (for a dtor, every subclass has one automatically), not that it has no body.

21.1.1 Example: A Horse Race

```
class HorseRace : public Subject {
    std::ifstream in;          // source of race results
    std::string lastWinner;
  public:
    HorseRace(const std::string &source) : in{source} {}
    bool runRace() { return bool(in >> lastWinner); }
    std::string getState() const { return lastWinner; }
};

class Bettor : public Observer {
    HorseRace *subject;
    std::string name, myHorse;
  public:
    Bettor(HorseRace *hr, const std::string &name, const std::string &horse)
        : subject{hr}, name{name}, myHorse{horse} {
        subject->attach(this);
    }
    ~Bettor() { subject->detach(this); }
    void notify() override {
        std::cout << name
                  << (subject->getState() == myHorse ? " wins!" : " loses.")
```

```
                    << std::endl;
    }
};

int main() {
    HorseRace hr{"races.txt"};
    Bettor larry{&hr, "Larry", "RunsLikeACow"};
    Bettor moe{&hr, "Moe", "Molasses"};
    while (hr.runRace()) hr.notifyObservers();
}
```

The subject knows nothing about bettors beyond the `Observer` interface, so new kinds of observers can be added without touching `HorseRace`.

21.2 The Decorator Pattern

We have an object with some behaviour, and while the program is running we want to add to (enhance) the behaviour of that object. Example: a window, to which we may add a menu, a scroll bar, a border, ... in any combination.

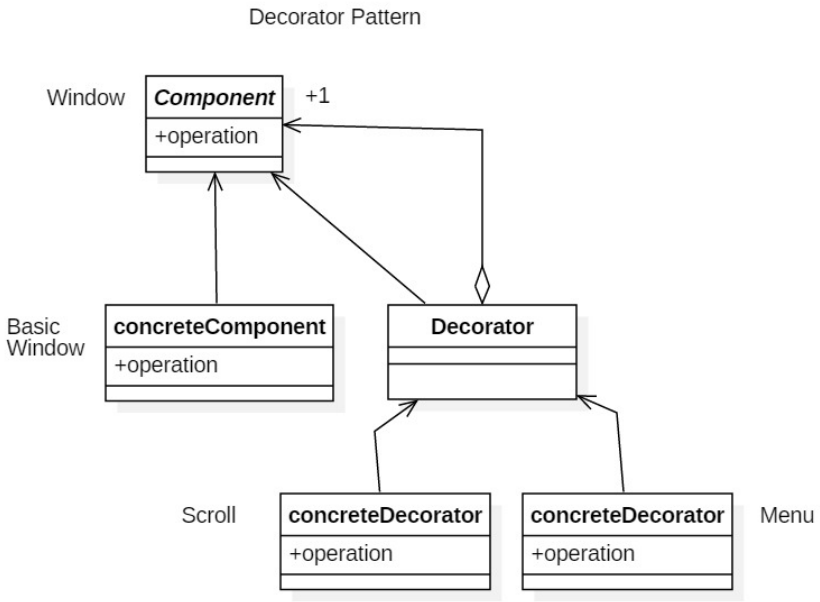


Figure 21.2: Decorator pattern (UML), with the window example: Window is the abstract component, BasicWindow the concrete component, Scroll and Menu concrete decorators.

- `Component`: the interface, with the operations to be enhanced.
- `ConcreteComponent`: the basic object.
- `Decorator`: an abstract class that **is-a** Component *and* **has-a** Component (the thing it decorates).
- `ConcreteDecorators`: each does its own bit and then delegates to the component it wraps.

```
Window *w = new BasicWindow{};
w = new Menu{w};        // a window with a menu
w = new Scroll{w};      // a window with a menu and a scroll bar
w->render();
```

Definition 21.1. Decorator pattern: add or enhance behaviour of an existing object at run time, by wrapping it in objects with the same interface.

21.2.1 Example: Pizza

```
class Pizza {                           // Component
  public:
    virtual float price() const = 0;
    virtual std::string desc() const = 0;
    virtual ~Pizza() {}
};

class CrustAndSauce : public Pizza {    // ConcreteComponent
  public:
    float price() const override { return 5.99; }
    std::string desc() const override { return "Pizza"; }
};

class Decorator : public Pizza {        // is-a Pizza
  protected:
    Pizza *component;                   // has-a Pizza
  public:
    Decorator(Pizza *p) : component{p} {}
    virtual ~Decorator() { delete component; }
};

class Topping : public Decorator {
    std::string theTopping;
  public:
    Topping(const std::string &t, Pizza *p) : Decorator{p}, theTopping{t} {}
    float price() const override { return component->price() + 0.75; }
    std::string desc() const override {
        return component->desc() + " with " + theTopping;
    }
};

class StuffedCrust : public Decorator {
  public:
    StuffedCrust(Pizza *p) : Decorator{p} {}
    float price() const override { return component->price() + 2.69; }
    std::string desc() const override {
        return component->desc() + " with stuffed crust";
    }
};

Pizza *p = new CrustAndSauce;
p = new Topping{"cheese", p};
```

```
p = new Topping{"mushrooms", p};
p = new StuffedCrust{p};
std::cout << p->desc() << " " << p->price() << std::endl;
// Pizza with cheese with mushrooms with stuffed crust 10.18
delete p;       // deletes the whole chain
```

Each decorator owns the component it wraps, so deleting the outermost one frees everything.

21.3 The Big Five Revisited (with Inheritance)

```
class Book {
    ...
  public:
    // defines the Big Five
};
class Text : public Book {
    std::string topic;
  public:
    // does not define the Big Five
};

Text b{"TheFlu", "Nomair", 50, "Medicine"};
Text c = b;     // calls Text's copy ctor
```

Remember you get one for free. The free copy ctor of `Text` calls Book's copy ctor on the Book part, then copies the `topic` field. Written out, the free versions are:

```
// copy ctor
Text::Text(const Text &other) : Book{other}, topic{other.topic} {}

// copy assignment: assign the Book part, then the subclass fields
Text &Text::operator=(const Text &other) {
    Book::operator=(other);
    topic = other.topic;
    return *this;
}
```

(`Book{other}` works because a Text is-a Book: `other` binds to `const Book&`.)

A first (wrong) attempt at the move ctor:

```
Text::Text(Text &&other) : Book{other}, topic{other.topic} {}   // incorrect
```

It won't crash, but it is *copying*, not stealing. (Quick check: if your move code looks the same as your copy code, it is wrong.)

```
int *p = ...;   // p is a pointer, and p (the variable) is an lvalue
```

Likewise, `other` has type "rvalue reference to Text", but `other` itself is a *named variable*, so it is an **lvalue**. Hence `Book{other}` calls Book's *copy* ctor and `topic{other.topic}` calls string's copy ctor.

We need a way to treat an lvalue as an rvalue: `std::move` (from `<utility>`) does exactly that — it is just a cast to `T&&`.

```
Text::Text(Text &&other)
    : Book{std::move(other)}, topic{std::move(other.topic)} {}

Text &Text::operator=(Text &&other) {
    Book::operator=(std::move(other));
    topic = std::move(other.topic);
    return *this;
}
```

(Using `other.topic` after `std::move(other)` is fine: Book's move ctor only steals the Book part.)

21.4 Assignment Through Base Class Pointers

```
Text b1{"TheFlu", "Nomair", 50, "Medicine"};
Text b2{"C++IsEasy", "Brad", 500, "CS"};
Book *pb1 = &b1;
Book *pb2 = &b2;
*pb1 = *pb2;     // object assignment through pointers
```

`Book::operator=` runs (it is non-virtual). Now `b1` is `{"C++IsEasy", "Brad", 500, "Medicine"}`: only the Book part was assigned. This is the **partial assignment problem**.

Possible solution: make `operator=` virtual?

```
class Book {
  public:
    virtual Book &operator=(const Book &other);
};
class Text : public Book {
  public:
    Text &operator=(const Text &other) override;  // does not compile:
};                                                // parameter types differ
```

To be a valid override, the parameter must be `const Book&`: `Text &operator=(const Book &other) override;`. But then *any* Book can be assigned to a Text: e.g. assigning a Comic to a Text through Book pointers would compile. Not all Books are Texts! This is the **mixed assignment problem**.

Alternative: disable assignment through base class pointers. Make Book's `operator=` non-public:

- private? Then subclasses can't call it either (they need it for their own `operator=`), so it must be `protected`.
- But Book is not abstract, so then one plain Book could never be assigned to another. That says the hierarchy was poorly designed to begin with.

The fix (next lecture): an abstract base class with a protected assignment operator, with plain books as one more concrete subclass.

Summary

- Observer: Subject keeps a list of `Observer*`, notifies them; a pure virtual dtor makes a class abstract but still needs a body.
- Decorator: is-a and has-a Component; wrap objects to add behaviour at run time.
- Subclass Big Five must handle the base part (`Book{other}`, `Book::operator=(other)`).
- Inside a move operation the rvalue reference is an lvalue: use `std::move` to keep moving.
- Assigning through base pointers causes partial assignment; a virtual `operator=` causes mixed assignment.

Chapter 22

Lecture 19: Abstract Base Classes, Factory Method, Template Method, NVI, Maps

March 15, 2018

22.1 Partial and Mixed Assignment: The Fix

```
Text a{...}, b{...};
Book *pa{&a};
Book *pb{&b};
*pa = *pb;      // partial assignment
```

Making `operator=` virtual forces `Text &Text::operator=(const Book &)`, which allows mixed assignment.

Solution. Restructure the hierarchy: make the base class abstract, give it a *protected* assignment operator, and make each kind of book (including plain "normal" books) a concrete subclass.

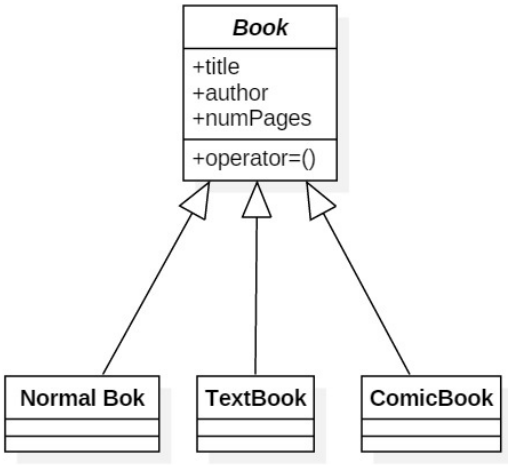


Figure 22.1: Abstract `Book` (italic name) with concrete subclasses `NormalBook`, `TextBook` and `ComicBook`.

```
class AbstractBook {
    std::string title, author;
    int numPages;
  protected:
    AbstractBook &operator=(const AbstractBook &other);  // subclasses only
  public:
    AbstractBook(...);
    virtual ~AbstractBook() = 0;     // abstract; still needs a body
};
AbstractBook::~AbstractBook() {}

class NormalBook : public AbstractBook {
  public:
    NormalBook &operator=(const NormalBook &other) {
        AbstractBook::operator=(other);
        return *this;
    }
};

class Text : public AbstractBook {
    std::string topic;
  public:
    Text &operator=(const Text &other) {
        AbstractBook::operator=(other);
        topic = other.topic;
        return *this;
    }
};
// Comic is similar
```

```
AbstractBook *pa1{&a};
AbstractBook *pb1{&b};
*pa1 = *pb1;    // won't compile: AbstractBook::operator= is protected
a = b;          // fine: Text::operator=, full assignment
```

Partial and mixed assignment through base pointers are now both impossible, while assignment between objects of the same concrete type still works.

Advice

In a class hierarchy, make superclasses abstract. (Non-leaf classes should be abstract.)

22.2 The Factory Method Pattern

Problem: write a video game with two kinds of enemies, turtles and bullets. The system randomly sends turtles and bullets, but bullets are more frequent in harder levels. We never know exactly which enemy comes next, so we can't call the Turtle or Bullet ctor directly; and the decision policy depends on the level.

```
class Level {
  public:
    virtual Enemy *createEnemy() = 0;   // the factory method
    virtual ~Level() {}
};

class Normal : public Level {
  public:
    Enemy *createEnemy() override {
        // more turtles (they are easier to kill)
    }
};

class Castle : public Level {
  public:
    Enemy *createEnemy() override {
        // more bullets
    }
};

Player *p = ...;
Level *l = new Normal;
Enemy *e = nullptr;
while (p->notDead()) {
    e = l->createEnemy();     // generate an enemy: the level decides which
    // attack the player
}
```

Also called the **Virtual Constructor pattern**: `createEnemy` acts like a constructor whose concrete class is chosen at run time. A related idea: make a class's constructors private/protected and provide methods that create objects for the client. We have seen this already: `List::addToFront` creates Nodes; `List::begin`/`List::end` create Iterators.

22.3 The Template Method Pattern

We want to allow subclasses to override *some* aspects of the superclass's behaviour, while other aspects must remain the same.

Example: red turtles and green turtles. Drawing a turtle means drawing a head, a shell and feet; only the shell differs.

```
class Turtle {
  public:
    void draw() {           // the template method: not virtual
        drawHead();
        drawShell();
        drawFeet();
    }
    virtual ~Turtle() {}
  private:
    void drawHead() { ... }
    void drawFeet() { ... }
    virtual void drawShell() = 0;   // the customizable step
};

class RedTurtle : public Turtle {
    void drawShell() override { /* draw a red shell */ }
};

class GreenTurtle : public Turtle {
    void drawShell() override { /* draw a green shell */ }
};
```

Subclasses can't change the overall drawing algorithm (`draw` is not virtual) but they can (must) supply the shell. Note that a subclass may override a private virtual method, even though it cannot call it.

22.4 The Non-Virtual Interface (NVI) Idiom

Think about what a **public virtual** method means:

- **public**: the class promises clients a certain, pre-defined behaviour (an interface, with pre/post-conditions);
- **virtual**: an invitation to subclasses to change that behaviour to anything they like.

These pull in opposite directions: we cannot guarantee the behaviour if subclasses can replace it.

NVI idiom

- All public methods should be non-virtual (except the destructor).
- All virtual methods should be private (or protected, if subclasses need to call the base version).

```
class DigitalMedia {         // not following NVI
  public:
    virtual void play() = 0;
    virtual ~DigitalMedia() {}
};
```

```
class DigitalMedia {          // using NVI
  public:
    void play() {
        checkCopyright();     // the base class keeps control:
        doPlay();             // e.g. copyright check before, stats after
    }
    virtual ~DigitalMedia() {}
  private:
    void checkCopyright();
    virtual void doPlay() = 0;
};
```

With NVI, the base class can later add code before/after every call (logging, checks) without changing any subclass. It costs nothing (`play` can be inlined), so the advice is to always use it. It generalizes the Template Method pattern: every virtual method is wrapped in a non-virtual template method.

22.5 The STL `map`

A `map` is a generalization of an array where the key (index) and the value can be of any type (the key type must support `operator<`, since maps are kept sorted — typically as balanced binary search trees). It is a template class parameterized on two types.

```
#include <map>
...
map<string, int> m;
m["abc"] = 1;
m["def"] = 4;
cout << m["abc"] << endl;   // prints 1
cout << m["ghi"] << endl;   // prints 0: if the key is absent, operator[]
                            // default-constructs a value and inserts it!

if (m.count("def")) { ... } // lookup without inserting:
                            // 0 if no such key, 1 if found
m.erase("abc");
```

Another STL class, `multimap`, allows several values per key — look at it on your own time.

Surprise, surprise: an iterator is predefined.

```
for (auto &p : m) {
    // p has type std::pair<const string, int> (a struct in the STL)
    cout << p.first << " " << p.second << endl;   // fields, not methods
}
```

Iteration happens in sorted key order.

22.6 The Visitor Pattern: Double Dispatch

The Visitor pattern is used for **double dispatch**: choosing which method to call based on the run-time types of *two* objects.

Example: enemies and weapons. The effect of striking depends on both the kind of enemy and the kind of weapon.

"Enenemy — that's not how you spell enemy." — Nomair

```
class Enemy {
  public:
    virtual void strike(Rock &) = 0;
    virtual void strike(Stick &) = 0;
};
```

This assumes we know the exact type of the weapon at compile time. Gameplay doesn't look like that:

```
Enemy *e = l->createEnemy();
Weapon *w = p->chooseWeapon();
e->strike(*w);      // which strike? *w is only known to be a Weapon
```

C++ dynamically dispatches on the object a method is called on, but not on the argument types (overloading is resolved at compile time).

The Visitor pattern is a clever combination of overriding and overloading:

```
class Enemy {
  public:
    virtual void strike(Weapon &w) = 0;
    virtual ~Enemy() {}
};

class Turtle : public Enemy {
  public:
    void strike(Weapon &w) override { w.useOn(*this); }  // *this is a Turtle
};
class Bullet : public Enemy {
  public:
    void strike(Weapon &w) override { w.useOn(*this); }  // *this is a Bullet
};

class Weapon {
  public:
    virtual void useOn(Turtle &t) = 0;
    virtual void useOn(Bullet &b) = 0;
    virtual ~Weapon() {}
};

class Stick : public Weapon {
  public:
    void useOn(Turtle &t) override { /* stick on turtle */ }
    void useOn(Bullet &b) override { /* stick on bullet */ }
};
class Rock : public Weapon {
  public:
    void useOn(Turtle &t) override { /* rock on turtle */ }
    void useOn(Bullet &b) override { /* rock on bullet */ }
};
```

The two `strike` bodies look identical, but putting them in `Enemy` to reuse code would be wrong: the whole point is that inside `Turtle`, the type of `*this` is known to be `Turtle`, which selects the right overload of `useOn`.

```
Enemy *e = ...;   // suppose it is a Turtle
Weapon *w = ...;  // suppose it is a Stick
e->strike(*w);
```

1. `strike` is virtual: `Turtle::strike` runs (dispatch on the enemy);
2. it calls `w.useOn(*this)` with `*this` of static type `Turtle&`: overload resolution picks `useOn(Turtle&)`;
3. `useOn` is virtual: `Stick::useOn(Turtle&)` runs (dispatch on the weapon).

More uses of Visitor next lecture.

Summary

- Fix partial/mixed assignment with an abstract base class whose `operator=` is protected; make non-leaf classes abstract.
- Factory Method: a virtual method creates objects whose concrete type the subclass decides.
- Template Method: a non-virtual algorithm calls virtual steps.
- NVI: public methods non-virtual, virtual methods private.
- `map<K,V>`: `operator[]` inserts defaults; use `count` to test membership; iteration yields sorted `pair`s.
- Visitor: `strike` (override) + `useOn(*this)` (overload) gives double dispatch.

Chapter 23

Lecture 20: Visitor Continued, Compilation Dependencies and Pimpl

March 20, 2018

> The original notes for this lecture are missing; this chapter reconstructs it from the course order (Visitor in Lecture 19, exception safety in Lecture 21) and the standard CS 246 sequence of that term.

23.1 Visitor Recap

Last time: double dispatch.

```
e->strike(*w);
// 1. virtual Enemy::strike   -> chooses on the enemy's run-time type
// 2. w.useOn(*this)          -> overload chosen by the static type of *this
// 3. virtual Weapon::useOn   -> chooses on the weapon's run-time type
```

23.2 Visitor, Second Use: Adding Operations to a Hierarchy

The Visitor pattern can also be used to add functionality to an existing class hierarchy *without* adding new virtual methods to every class.

Suppose we want to do something with our Book hierarchy that its designers did not anticipate — e.g. build a catalogue: count Books by author, Texts by topic, Comics by hero. Each new operation would normally mean a new virtual method in `Book`, `Text` and `Comic`. Instead, add a single `accept` method once, and let *visitors* supply the operations:

```
class BookVisitor {
  public:
    virtual void visit(Book &b) = 0;
    virtual void visit(Text &t) = 0;
    virtual void visit(Comic &c) = 0;
    virtual ~BookVisitor() {}
};

class Book {
```

```
  public:
    virtual void accept(BookVisitor &v) { v.visit(*this); }
    virtual ~Book() {}
    ...
};
class Text : public Book {
  public:
    void accept(BookVisitor &v) override { v.visit(*this); }
    ...
};
class Comic : public Book {
  public:
    void accept(BookVisitor &v) override { v.visit(*this); }
    ...
};
```

(`accept` plays the role of `strike`; `visit` plays the role of `useOn`.)

A visitor that builds the catalogue:

```
class Catalogue : public BookVisitor {
    std::map<std::string, int> theCatalogue;
  public:
    std::map<std::string, int> getCatalogue() const { return theCatalogue; }
    void visit(Book &b) override { ++theCatalogue[b.getAuthor()]; }
    void visit(Text &t) override { ++theCatalogue[t.getTopic()]; }
    void visit(Comic &c) override { ++theCatalogue[c.getHero()]; }
};

std::vector<Book*> collection{...};
Catalogue cat;
for (auto b : collection) b->accept(cat);
for (auto &p : cat.getCatalogue())
    std::cout << p.first << ": " << p.second << std::endl;
```

Recall that `m[key]` inserts `0` for a missing key, so `++theCatalogue[...]` counts correctly from zero.

Any further operation (e.g. total pages per category, printing a report) is just another `BookVisitor` subclass; `Book`, `Text` and `Comic` are untouched.

Trade-off of Visitor

Visitor makes it easy to add new *operations*, but hard to add new *classes* to the visited hierarchy: a new Book subclass requires a new `visit` overload in `BookVisitor` and in every visitor.

23.3 Compilation Dependencies

Visitor creates a lot of mutual dependencies: `book.h` mentions `BookVisitor`, and `bookvisitor.h` mentions `Book`, `Text` and `Comic`. If each header `#include`s the other we get an include cycle, and the include guards cause one of the files to see an undefined class.

Q. When do we need `#include "a.h"`, and when does a **forward declaration** `class A;` suffice?

The compiler needs the full definition of a class when it needs to know the class's *size* or its *members*:

```
// a.h
class A { ... };

// b.h
#include "a.h"
class B : public A { ... };      // inheritance: needs A's size and members

// c.h
#include "a.h"
class C {
    A myA;                       // a field of type A: needs A's size
};

// d.h
class A;                         // forward declaration is enough
class D {
    A *myAp;                     // pointers all have the same size
};

// e.h
class A;
class E {
    A f(A x);                    // only a declaration: no size needed yet
};
```

In the implementation files, D and E will need the full definition of A (`d.cc` dereferences the pointer; `e.cc` actually passes and returns A's), so `d.cc` and `e.cc` `#include "a.h"`.

Advice

Do not `#include` a header when a forward declaration suffices. Prefer forward declarations in headers and includes in `.cc` files.

Why bother?

- it breaks include cycles;
- it reduces compilation dependencies: if `a.h` changes, only files that really include it are recompiled (`make` with `-MMD` follows the includes). Large projects can take hours to build, so this matters.

23.4 The Pimpl Idiom

Even a private field creates a compilation dependency:

```
// window.h
#include <X11/Xlib.h>
class XWindow {
    Display *d;
    Window w;
    int s;
```

```
    GC gc;
    unsigned long colours[10];
  public:
    ...
};
```

Every client that includes `window.h` also includes the whole X11 header, and must be recompiled whenever a *private* field of `XWindow` is added or changed — even though clients can't use those fields.

Definition 23.1. Pimpl ("pointer to implementation"): move all private fields into a separate implementation struct, defined only in the `.cc` file, and store only a pointer to it in the class.

```
// window.h
struct XWindowImpl;              // forward declaration only
class XWindow {
    XWindowImpl *pImpl;
  public:
    XWindow();
    ~XWindow();
    ...                          // no change to the public interface
};

// windowimpl.h
#include <X11/Xlib.h>
struct XWindowImpl {
    Display *d;
    Window w;
    int s;
    GC gc;
    unsigned long colours[10];
};

// window.cc
#include "window.h"
#include "windowimpl.h"
XWindow::XWindow() : pImpl{new XWindowImpl} { ... }
XWindow::~XWindow() { delete pImpl; }
// other methods use pImpl->d, pImpl->w, ... instead of d, w, ...
```

Now clients include only `window.h`, which does not include X11. If the implementation struct changes, only `window.cc` is recompiled; clients just re-link. (Since the class now owns heap memory, the Rule of Five applies; with a smart pointer from Lecture 21, `std::unique_ptr<XWindowImpl>`, most of it is automatic.)

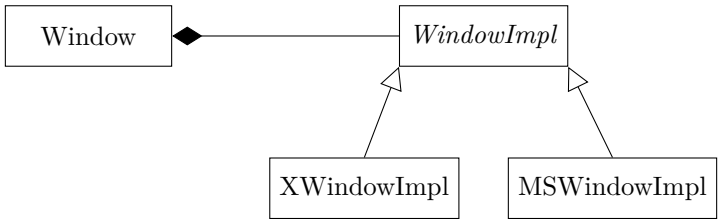
23.5 The Bridge Pattern

Taking Pimpl further: if there are several possible window implementations (X11, a Windows API, a test stub), make the implementation struct a *polymorphic class hierarchy*:

```
class WindowImpl {               // abstract implementation interface
  public:
    virtual void drawPixel(int x, int y, int colour) = 0;
```

```
    virtual ~WindowImpl() {}
};
class XWindowImpl : public WindowImpl { ... };
class MSWindowImpl : public WindowImpl { ... };

class Window {                    // abstraction
    WindowImpl *pImpl;            // chosen at construction time
  public:
    void fillRectangle(int x, int y, int w, int h, int colour);  // uses pImpl
};
```



Definition 23.2. Bridge pattern: separate an abstraction from its implementation so the two can vary independently — new kinds of windows (subclassing `Window`) and new platforms (subclassing `WindowImpl`) can each be added without touching the other hierarchy.

Summary

- Visitor also adds operations to a hierarchy: one `accept` per class, one visitor class per operation. Easy to add operations, hard to add classes.
- Include a header only when you need a class's size or members; otherwise forward-declare. This breaks cycles and speeds up rebuilds.
- Pimpl hides private fields behind a pointer to an implementation struct, so clients don't recompile when the implementation changes.
- Bridge: Pimpl with a polymorphic implementation hierarchy.

Chapter 24

Lecture 21: Exception Safety, RAII and Smart Pointers

March 22, 2018

24.1 The Problem

```
void f() {
    MyClass *p = new MyClass;
    MyClass mc;
    g();
    delete p;
}
```

Suppose `g` itself does not leak memory. Still, if `g` throws an exception, `delete p` is never executed: `f` leaks memory.

We want to write **exception-safe** code. (This does *not* mean "code that doesn't use exceptions".) Exceptions are meant to be recovered from, so an exception should not leave behind

- memory leaks,
- dangling pointers,
- broken class invariants.

C++ guarantee

During stack unwinding, all stack-allocated objects are destroyed: their destructors run and their memory is reclaimed. Nothing is promised about heap memory.

So in `f`, `mc` is cleaned up, but the object `p` points to leaks. A pointer is just a value: destroying `p` does not (and should not, on its own) call `delete` on it.

24.2 Refactoring by Hand

```
void f() {
    MyClass *p = new MyClass;
    try {
        MyClass mc;
        g();
    } catch (...) {
        delete p;
        throw;          // re-throw: we only cleaned up
    }
    delete p;
}
```

This is tedious and error-prone (note the duplicated cleanup, and imagine five pointers). We would like the ability to run some code no matter whether an exception occurred:

- Java has `finally`;
- Scheme has `dynamic-wind`.

C++ has neither — and doesn't need one, because of the guarantee above: the only code guaranteed to run during unwinding is the destructors of stack objects. So:

1. maximize your use of the stack;
2. but what if you really do need the heap?

24.3 RAII

Definition 24.1. RAII — *Resource Acquisition Is Initialization* (a C++ idiom named by Bjarne Stroustrup): every resource should be wrapped inside a stack-allocated object whose destructor's job is to release the resource.

We have been using it since the second C++ lecture:

```
{
    ifstream f{"hello.txt"};   // file is opened (acquired) at initialization
    ...
}                              // f's dtor runs: the file is closed (released)
```

The file is closed whether the block ends normally or by an exception.

Heap memory is a resource! Idea: wrap the heap object inside a stack object, whose destructor deletes the heap object.

24.4 `std::unique_ptr`

The standard library (`<memory>`) provides the template class `std::unique_ptr<T>`:

- its ctor takes a `T*`;
- its dtor calls `delete` on that pointer;
- it overloads `*` and `->`, so it can be used exactly like the raw pointer to the heap object.

```
#include <memory>

void f() {
    std::unique_ptr<MyClass> p{new MyClass};
    MyClass mc;
    g();
}   // no leak, whether or not g throws
```

Better: use `std::make_unique`, which allocates the object and passes its arguments to the `MyClass` ctor:

```
auto p = std::make_unique<MyClass>(/* args to MyClass ctor, if any */);
```

What about copying?

```
unique_ptr<C> p{new C};      // or: auto p = make_unique<C>();
unique_ptr<C> q = p;         // won't compile
```

If both `p` and `q` pointed to the object, it would be deleted twice. So copying is disabled for `unique_ptr`; moving still works (stealing is still acceptable in C++ society):

```
unique_ptr<C> q = std::move(p);   // q owns the object; p is now null
```

24.4.1 A Basic Implementation

```
template <typename T> class unique_ptr {
    T *ptr;
  public:
    explicit unique_ptr(T *p) : ptr{p} {}
    ~unique_ptr() { delete ptr; }

    unique_ptr(const unique_ptr &other) = delete;              // no copy ctor
    unique_ptr &operator=(const unique_ptr &other) = delete;   // no copy assign

    unique_ptr(unique_ptr &&other) : ptr{other.ptr} {          // move ctor
        other.ptr = nullptr;
    }
    unique_ptr &operator=(unique_ptr &&other) {                // move assign
        using std::swap;
        swap(ptr, other.ptr);
        return *this;
    }

    T &operator*() const { return *ptr; }
    T *operator->() const { return ptr; }
    T *get() const { return ptr; }
};
```

`= delete` explicitly removes a function the compiler would otherwise provide for free. (Note the move operations take `unique_ptr&&`, not `const unique_ptr&&`: we must modify `other` to steal from it.)

24.4.2 Ownership

With smart pointers, the type of a pointer documents ownership:

- `unique_ptr<T>`: **owns** the object (responsible for deleting it). Composition (owns-a) relationships use `unique_ptr` fields — and then the class needs no hand-written destructor.
- raw pointer `T*`: **does not own**, just observes. Aggregation (has-a) relationships use raw pointers (or references). Get one from a `unique_ptr` with `p.get()`.
- Never call `delete` on a raw pointer in a program that uses smart pointers for ownership.

24.5 `std::shared_ptr`

Sometimes there is no single owner: several objects legitimately share one heap object, and it should be deleted when the *last* of them is done with it.

```
void f() {
    auto p1 = std::make_shared<C>();
    if (...) {
        auto p2 = p1;   // copy ctor: p1 and p2 point to the same heap object
        ...
    }                   // p2 goes out of scope: dtor runs, object NOT deleted
    ...
}                       // p1 goes out of scope: dtor runs, object deleted
```

Shared pointers use **reference counting**: a counter (stored on the heap next to the object) records how many `shared_ptr`s point to the same object. Copying increments the count; destroying (or reassigning) a `shared_ptr` decrements it; when the count reaches 0 the object is deleted.

Caveats

- Reference counting costs time and space; prefer `unique_ptr` unless ownership really is shared.
- **Cycles**: if A holds a `shared_ptr` to B and B holds a `shared_ptr` to A, neither count ever reaches 0 and both leak. Break cycles by making one direction a non-owning pointer (raw pointer or `std::weak_ptr`, which observes without contributing to the count).
- Never create two separate `shared_ptr`s from the same raw pointer (`shared_ptr<C> a{p}, b{p};`): they keep separate counts and both delete `p`. Use `make_shared`, and copy the `shared_ptr` itself.

Summary

- Exception safety: no leaks, dangling pointers or broken invariants when an exception passes through.
- C++ only guarantees that stack objects are destroyed during unwinding, so wrap every resource in a stack object (RAII).

- `unique_ptr`: sole owner, move-only; create with `make_unique`.
- `shared_ptr`: shared ownership via reference counting; create with `make_shared`; beware cycles.
- Raw pointers are for non-owning access.

Chapter 25

Topic: Exception Safety Guarantees

This chapter continues Lecture 21 and covers material from the remainder of the term (not in the original lecture notes).

25.1 Levels of Exception Safety

Lecture 21 made sure resources are released when an exception passes through. But what state is an object left in? There are three standard levels of guarantee a function can offer:

Definition 25.1.

1. **Basic guarantee**: if an exception occurs, the program is in *some* valid but unspecified state. Nothing is leaked, no invariants are broken.
2. **Strong guarantee**: if an exception is raised while executing `f`, the state of the program is as if `f` had not been called ("commit or roll back").
3. **No-throw guarantee**: `f` never throws an exception, and always accomplishes its task.

A function that gives none of these is not exception safe at all.

25.2 An Example

```
class A { ... };
class B { ... };
class C {
    A a;
    B b;
  public:
    void f() {
        a.g();   // may throw (strong guarantee)
        b.h();   // may throw (strong guarantee)
    }
};
```

Is `C::f` exception safe?

- If `a.g()` throws: nothing has happened yet (`g` has the strong guarantee). OK.

- If `b.h()` throws: the effects of `h` are undone, but `a.g()` has already happened and is not undone. So `f` would need to undo `g`'s effects — very hard or impossible if `g` had non-local side effects (printing, writing a file).

So `f` offers only the basic guarantee (at best).

25.2.1 Achieving the Strong Guarantee

If `A::g` and `B::h` have no non-local side effects, we can do the work on *copies* and then swap them in:

```
class C {
    A a;
    B b;
  public:
    void f() {
        A atemp = a;     // may throw: fine, nothing changed yet
        B btemp = b;
        atemp.g();       // may throw: fine, only the copies changed
        btemp.h();
        a = atemp;       // what if copy assignment throws?
        b = btemp;
    }
};
```

The last two lines are the problem: if `b = btemp` throws, `a` has already been modified. We need the "commit" step to be no-throw. Assigning *pointers* can't throw — so use the pImpl idiom:

```
struct CImpl {
    A a;
    B b;
};

class C {
    std::unique_ptr<CImpl> pImpl;
  public:
    void f() {
        auto temp = std::make_unique<CImpl>(*pImpl);  // copy: may throw, fine
        temp->a.g();                                  // may throw, fine
        temp->b.h();                                  // may throw, fine
        std::swap(pImpl, temp);                       // no-throw commit
    }   // temp (the old state) is destroyed here
};
```

Now `f` has the strong guarantee (assuming `g` and `h` have no non-local side effects). This is the same idea as copy-and-swap for `operator=`.

Rule of thumb

Do everything that might throw on the side (on copies/temporaries), then commit with operations that cannot throw (pointer assignment, swap of built-in types, moves marked `noexcept`).

25.3 Exception Safety and `vector`

`vector` manages a heap array with RAII: when a vector goes out of scope, its dtor frees the array and calls the dtor of every element.

- `vector<C>`: owns the `C` objects; they are destroyed with the vector.
- `vector<C*>`: the dtor frees the array of pointers, but does *not* delete the objects pointed to (pointers have no dtor). If the vector should own them, you must delete them yourself:

```
for (auto p : v) delete p;
```

- `vector<unique_ptr<C>>`: owns the objects, and frees them automatically, even during unwinding. No explicit cleanup loop needed.

25.3.1 `emplace_back` and `noexcept`

`vector::emplace_back` (and `push_back`) offer the strong guarantee: if it throws, the vector is unchanged. When the array is full, it must

1. allocate a bigger array,
2. copy the elements from the old array to the new one,
3. delete the old array.

If a copy throws midway (step 2), the new array is simply discarded and the old one is intact: strong guarantee.

But copying is expensive; moving would be much faster. However, if a *move* throws midway, some elements of the old array have already been gutted, and the original state can't be restored. So `vector` only moves elements if the move ctor is guaranteed not to throw; otherwise it copies.

Advice

Declare move constructors and move assignment operators (and `swap`) as `noexcept` when they cannot throw — which is almost always.

```
class C {
  public:
    C(C &&other) noexcept;
    C &operator=(C &&other) noexcept;
};
```

Otherwise `vector` (and other library code) will silently fall back to the slower copy. Destructors are implicitly `noexcept`.

25.4 Exception-Safe `operator=`

Compare the versions of `Node::operator=` from Lectures 10–11:

Version	Guarantee
`delete next;` then `new`	none: leaves a dangling `next` if `new` throws
`new` into temp, then `delete` old	strong
copy-and-swap	strong (if the copy ctor is safe and `swap` is no-throw)

Summary

- Basic: valid state, no leaks. Strong: all-or-nothing. No-throw: never fails.
- A sequence of strong operations is not automatically strong.
- Achieve strong guarantee by working on copies and committing with no-throw operations (pImpl + swap).
- Use `vector<unique_ptr<C>>` for owned polymorphic objects.
- Mark moves and swaps `noexcept` so containers can use them.

Chapter 26

Topic: Coupling, Cohesion and MVC

Material from the remainder of the term, not covered in the original lecture notes.

26.1 Measures of Design Quality

How do we tell whether a modular design is good? Two classic measures.

26.1.1 Coupling

Definition 26.1. Coupling is how much distinct modules (classes) depend on each other.

From low (good) to high (bad):

1. modules communicate via function calls with basic parameters/results;
2. modules pass arrays/structs back and forth;
3. modules affect each other's control flow (e.g. passing flags that tell the other module what to do);
4. modules share global data;
5. modules have access to each other's implementation (`friend`s, public fields).

With high coupling, changes to one module require changes to others, and modules are hard to reuse individually.

26.1.2 Cohesion

Definition 26.2. Cohesion is how closely the elements of a single module are related to each other.

From low (bad) to high (good):

1. arbitrary grouping of unrelated elements (e.g. `<utility>`: `swap`, `pair`, `move`, ...);
2. elements share a common theme but are otherwise unrelated, maybe sharing some base code (e.g. `<algorithm>`);

3. elements manipulate the same state over the lifetime of an object (e.g. open/read/close a file);
4. elements pass data to each other (a pipeline);
5. elements cooperate to perform exactly one task.

With low cohesion, modules are poorly organized and hard to understand and maintain.

Goal

Low coupling, high cohesion.

26.2 Decoupling the Interface

Your primary program classes should not be printing things. Consider

```
class ChessBoard {
    ...
    void move(...) {
        ...
        std::cout << "Your move" << std::endl;
    }
};
```

This is bad design: it inhibits code reuse. What if we want a graphical display, or to use the same game logic over the network, or to write output to a file? Slightly better: give the class a stream to print to,

```
class ChessBoard {
    std::ostream &out;
  public:
    ChessBoard(std::ostream &out) : out{out} {}
    ...
};
```

but it still assumes a text interface.

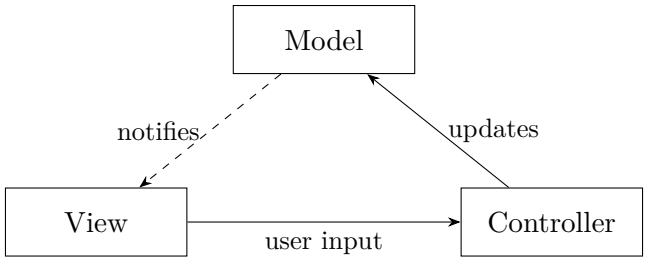
Definition 26.3. Single Responsibility Principle: a class should have only one reason to change.

If two distinct parts of the specification can change independently (the rules of chess vs. how the board is displayed), they belong in different classes. `ChessBoard` should communicate with its user via parameters, results, and possibly exceptions; a separate class handles I/O.

26.3 Model–View–Controller (MVC)

Definition 26.4. MVC separates a program into three parts:

- the **model**: the main data and logic you are manipulating (e.g. the game state);
- the **view**: how the model is displayed;
- the **controller**: how the model is manipulated (interprets user input, decides what to do).



- The model may have multiple views (text and graphics at the same time). It doesn't need to know their details: the model is typically a **subject** and the views are **observers** (Observer pattern, Lecture 18).
- The controller mediates control flow between model and view. The model may (or may not) hold a reference to the controller.
- Swapping out the view or controller does not require changing the model, and the model can be tested without any UI.

```
class Game : public Subject {          // model
    Board board;
  public:
    bool makeMove(const Move &m);      // returns false if illegal
    const Board &getState() const { return board; }
};

class TextDisplay : public Observer {  // view
    Game &game;
    std::ostream &out;
  public:
    TextDisplay(Game &g, std::ostream &out) : game{g}, out{out} {
        game.attach(this);
    }
    void notify() override { out << game.getState(); }
};

class Controller {                     // controller
    Game &game;
  public:
    explicit Controller(Game &g) : game{g} {}
    void play(std::istream &in) {
        Move m;
        while (in >> m) {
            if (game.makeMove(m)) game.notifyObservers();
            else std::cerr << "Illegal move" << std::endl;
        }
    }
};
```

26.4 Other Design Principles

- **Open/Closed Principle**: classes should be open for extension but closed for modification. Changes in a program should happen by writing *new* code (new subclasses, new decorators, new visitors), not by editing working code. Strategies, Decorator, Template Method and Visitor all support this.
- **Liskov Substitution Principle**: if B is a subtype of A, a B must be usable anywhere an A is expected, without the client noticing. Subclasses must not strengthen preconditions, weaken postconditions or break invariants of the superclass. (Classic violation: `Square` as a subclass of a mutable `Rectangle` — setting the width of a square also changes its height, which no Rectangle client expects.)
- **Program to an interface** (Lecture 17), and prefer composition over inheritance when the relationship is not really is-a.

Summary

- Aim for low coupling (modules depend on each other as little as possible) and high cohesion (each module does one coherent thing).
- Keep I/O out of core logic (Single Responsibility).
- MVC: model (data/logic, a Subject), views (Observers that display it), controller (turns input into model operations).
- Open/Closed and Liskov Substitution guide good use of inheritance.

Chapter 27

Topic: Template Functions, STL Iterators and Algorithms

Material from the remainder of the term, not covered in the original lecture notes. Builds on templates (Lecture 16), `vector` and `map` (Lectures 16 and 19), and the abstract iterator (Lecture 17).

27.1 Template Functions

Functions can be templates too:

```
template <typename T> T min(T x, T y) {
    return x < y ? x : y;
}

int a = 1, b = 2;
int c = min(a, b);          // T = int, deduced from the arguments
double d = min(1.5, 2.5);   // T = double
int e = min<int>(1, 2.5);   // T given explicitly (2.5 converted to int)
```

The compiler deduces `T` from the argument types. The template only works for types `T` that support everything the body uses (here `operator<` and copying); otherwise instantiation fails to compile.

Recall `foreach` from Lecture 17, which required our own `AbsIter`. Templates make it fully general, with no inheritance at all:

```
template <typename Iter, typename Fn>
void foreach(Iter start, Iter finish, Fn f) {
    while (start != finish) {
        f(*start);
        ++start;
    }
}

int a[] = {1, 2, 3, 4, 5};
foreach(a, a + 5, addFive);    // raw pointers are iterators!
vector<int> v{1, 2, 3};
foreach(v.begin(), v.end(), addFive);
```

`Iter` can be any type supporting `!=`, `*` and prefix `++` — including plain pointers. This is how the STL algorithms work.

27.2 Iterator Categories

Different containers support different iterator operations:

Category	Supports	Example
input	read `*it`, `++`, single pass	`istream_iterator`
output	write `*it = x`, `++`	`ostream_iterator`
forward	read/write, multi-pass, `++`	`forward_list`
bidirectional	forward + `--`	`list`, `map`, `set`
random access	bidirectional + `it + n`, `it[n]`, `<`	`vector`, pointers

Algorithms state which category they need; e.g. `sort` needs random access, so it works on a `vector` but not on a `list`.

27.3 The `<algorithm>` Library

A suite of template functions, most of which work over iterator ranges `[start, finish)` (start included, finish excluded).

```
#include <algorithm>

// for_each: apply f to every element
template <typename Iter, typename Fn>
Fn for_each(Iter start, Iter finish, Fn f);

// find: iterator to the first element == val, or finish if none
template <typename Iter, typename T>
Iter find(Iter first, Iter last, const T &val) {
    for (; first != last; ++first)
        if (*first == val) return first;
    return last;
}

// count: number of elements == val
// copy: copy [first, last) to the range starting at result
template <typename InIter, typename OutIter>
OutIter copy(InIter first, InIter last, OutIter result) {
    while (first != last) *result++ = *first++;
    return result;
}

// transform: result[i] = f(first[i])
template <typename InIter, typename OutIter, typename Fn>
OutIter transform(InIter first, InIter last, OutIter result, Fn f) {
    while (first != last) *result++ = f(*first++);
    return result;
}
```

copy does not allocate

`copy` and `transform` *assume* the destination has room:

```
vector<int> v{1, 2, 3, 4, 5, 6, 7};
vector<int> w(4);                                   // 4 zeros: room for 4
copy(v.begin() + 1, v.begin() + 5, w.begin());      // w = {2, 3, 4, 5}
```

Copying into an empty vector writes past its end: undefined behaviour.

27.4 Function Objects

The parameter `Fn f` can be anything that can be called with `()`: a function, or an object of a class that overloads `operator()`.

```
class Plus1 {
  public:
    int operator()(int n) const { return n + 1; }
};

Plus1 p;
p(4);     // 5: p looks like a function

transform(v.begin(), v.end(), w.begin(), Plus1{});
```

Unlike functions, function objects can carry state:

```
class Plus {
    int m;
  public:
    explicit Plus(int m) : m{m} {}
    int operator()(int n) const { return n + m; }
};
transform(v.begin(), v.end(), w.begin(), Plus{5});

class IncreasingPlus {          // state changes between calls
    int m = 0;
  public:
    int operator()(int n) { return n + (m++); }
};
// v = {1, 1, 1} becomes w = {1, 2, 3}
transform(v.begin(), v.end(), w.begin(), IncreasingPlus{});
```

27.5 Lambdas

Writing a class for every little function is tedious. A **lambda** expression creates an anonymous function object inline:

```
// how many ints in v are even?
bool even(int n) { return n % 2 == 0; }
int x = count_if(v.begin(), v.end(), even);

int y = count_if(v.begin(), v.end(), [](int n) { return n % 2 == 0; });
```

Syntax: `[captures](parameters) { body }`. The capture list says which local variables the lambda may use:

```
int m = 5;
transform(v.begin(), v.end(), w.begin(), [m](int n) { return n + m; });  // by value
int total = 0;
for_each(v.begin(), v.end(), [&total](int n) { total += n; });          // by ref
```

`[=]` captures everything used by value, `[&]` by reference.

27.6 Iterator Adaptors

`<iterator>` provides iterators that don't walk over a container:

```
#include <iterator>

// write to a stream: each assignment *it = x prints x followed by ", "
ostream_iterator<int> out{cout, ", "};
copy(v.begin(), v.end(), out);          // prints 1, 2, 3, 4, 5, 6, 7,

// grow a container: each assignment does push_back
vector<int> w;                          // empty!
copy(v.begin(), v.end(), back_inserter(w));   // fine: w grows as needed
```

`back_inserter` fixes the "`copy` doesn't allocate" problem.

27.7 More Containers

Container	Notes
`vector<T>`	contiguous array; $\mathcal{O}(1)$ amortized `push_back`; $\mathcal{O}(n)$ insert/erase in the middle
`deque<T>`	like vector, plus $\mathcal{O}(1)$ `push_front`
`list<T>`	doubly linked list; $\mathcal{O}(1)$ insert/erase anywhere given an iterator; no random access
`map<K,V>`	sorted by key (balanced BST); $\mathcal{O}(\log n)$ operations
`set<T>`	sorted unique elements
`unordered_map<K,V>`	hash table; expected $\mathcal{O}(1)$ operations; unordered

27.8 Example

Read words from stdin, and print each distinct word with its frequency, sorted:

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    map<string, int> freq;
    string w;
    while (cin >> w) ++freq[w];
```

```
    for (auto &p : freq) cout << p.first << " " << p.second << endl;
}
```

Summary

- Template functions deduce their type parameters from the arguments.
- STL algorithms work on iterator ranges `[first, last)` and any type with the right operations, including pointers.
- Iterator categories determine which algorithms apply.
- Pass behaviour as functions, function objects (`operator()`), or lambdas, written `[captures](params){body}`.
- `copy`/`transform` don't allocate; use `back_inserter` or size the destination first; `ostream_iterator` prints.

Chapter 28

Topic: Casting and Run-Time Type Information

Material from the remainder of the term, not covered in the original lecture notes.

28.1 C-Style Casts

In C:

```
Node n;
int *ip = (int *) &n;   // forces C to treat a Node* as an int*
```

C-style casts should be avoided in C++. They do several very different things with the same syntax, they are hard to search for, and the compiler won't stop you from doing something meaningless. If you must cast, use one of the four C++ casts, which say exactly what you mean.

28.2 `static_cast`

For "sensible" casts with well-defined semantics, checked at compile time:

```
double d = 3.7;
void f(int x);
void f(double x);
f(static_cast<int>(d));    // calls f(int); d is truncated to 3
```

Downcasting a superclass pointer to a subclass pointer:

```
Book *b = new Text{...};
Text *t = static_cast<Text*>(b);
t->getTopic();
```

You are telling the compiler "trust me, `b` really points to a Text". If you are wrong, the behaviour is undefined. `static_cast` does no run-time check.

28.3 `reinterpret_cast`

For unsafe, implementation-dependent "weird" conversions: reinterpret the bits of one type as another.

```
Student s;
Turtle *t = reinterpret_cast<Turtle*>(&s);   // treat a Student as a Turtle
```

Almost any use is undefined or non-portable. You will rarely, if ever, need it.

28.4 const_cast

For converting between `const` and non-`const`; the only C++ cast that can "cast away" const.

```
void g(int *p);               // suppose g doesn't actually modify *p,
                              // but its author forgot the const
void f(const int *p) {
    g(const_cast<int*>(p));   // OK as long as g really doesn't modify *p
}
```

Useful for working with libraries that are not const-correct. Modifying an object that was originally declared const through such a pointer is undefined behaviour.

28.5 dynamic_cast

Is it safe to convert a `Book*` to a `Text*`? It depends on what the pointer actually points to. `dynamic_cast` checks at run time:

```
Book *pb = ...;
Text *pt = dynamic_cast<Text*>(pb);
if (pt) cout << pt->getTopic();   // pb really pointed to a Text
else cout << "Not a Text";        // dynamic_cast returned nullptr
```

- If the cast works (`*pb` is a Text, or a subclass of Text), `pt` points to the object; otherwise `pt` is `nullptr`.
- `dynamic_cast` only works on classes with at least one virtual method (it uses the vtable to find the run-time type — see the next chapter). This information is called **RTTI** (run-time type information).
- With references: there's no null reference, so a failing `dynamic_cast<Text&>(b)` throws `std::bad_cast`.

28.5.1 Casting Smart Pointers

```
shared_ptr<Book> sb = make_shared<Text>(...);
shared_ptr<Text> st = dynamic_pointer_cast<Text>(sb);   // shares ownership
// also: static_pointer_cast, const_pointer_cast
```

28.6 Using RTTI: Polymorphic Assignment Revisited

Dynamic casting lets us solve the mixed-assignment problem from Lecture 18 with a virtual `operator=`:

```
Text &Text::operator=(const Book &other) {     // virtual, overrides Book's
    const Text &tother = dynamic_cast<const Text&>(other);  // throws bad_cast
    if (this == &tother) return *this;                      // if not a Text
    Book::operator=(other);
    topic = tother.topic;
    return *this;
}
```

Assigning a Comic to a Text through Book references now throws instead of silently corrupting the Text.

28.7 Is Dynamic Casting Good Style?

```
void whatIsIt(Book *b) {
    if (dynamic_cast<Comic*>(b)) cout << "Comic";
    else if (dynamic_cast<Text*>(b)) cout << "Text";
    else if (b) cout << "Book";
    else cout << "Nothing";
}
```

This is tightly coupled to the Book hierarchy: every time a new subclass is added, this function (and every function like it) must be found and changed. It is exactly what virtual methods are for:

```
class Book {
  public:
    virtual std::string identify() const { return "Book"; }
};
// each subclass overrides identify()
```

When a hierarchy is *closed* (the set of subclasses is fixed) but the set of operations is open, Visitor (Lecture 20) is the virtual-method-based answer. Code that switches on run-time type with `dynamic_cast` is a design smell: use virtual methods (or Visitor) whenever you can.

Summary

Cast	Use
`static_cast`	well-defined conversions; unchecked downcasts
`reinterpret_cast`	reinterpret bits; unsafe, non-portable
`const_cast`	add/remove `const`
`dynamic_cast`	checked downcast using RTTI; `nullptr` or `bad_cast` on failure

Avoid C-style casts; prefer virtual methods to type switches built from `dynamic_cast`.

Chapter 29

Topic: How Virtual Methods Work (Vtables and Object Layout)

Material from the remainder of the term, not covered in the original lecture notes.

29.1 The Question

```
class Vec {
    int x, y;
    void f();
};

class Vec2 {
    int x, y;
    virtual void f();
};

Vec v{1, 2};
Vec2 w{1, 2};
cout << sizeof(v) << " " << sizeof(w);   // ?
```

On a typical 64-bit machine this prints `8 16`. Both have two `int`s (8 bytes); methods are not stored in objects at all (there is one copy of each method's code). So why is `w` bigger? Because of the virtual method.

29.2 Dynamic Dispatch Needs Run-Time Information

```
Book *pb = (userSaysComic ? new Comic{...} : new Book{...});
pb->isHeavy();   // Book::isHeavy or Comic::isHeavy?
```

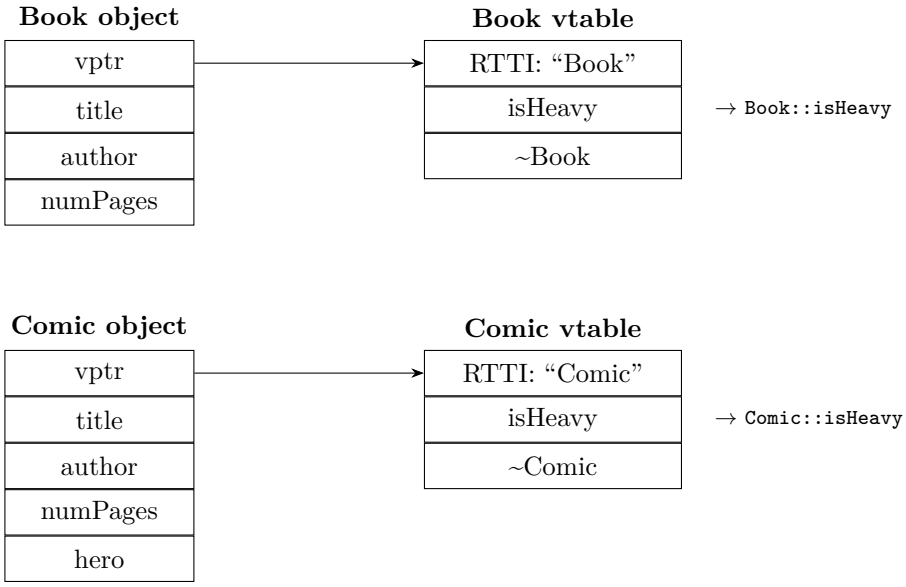
The compiler can't know which method to call; the decision must be made at run time, so the *object* has to carry enough information to find the right method.

29.3 Vtables and Vptrs

For each class with virtual methods, the compiler creates one **virtual table** (**vtable**): a static array of pointers to the class's virtual methods (plus the RTTI used by `dynamic_cast`).

Each object of such a class contains a hidden pointer, the **vptr**, to its class's vtable. The vptr is set by the constructor.

```
class Book {
    std::string title, author;
    int numPages;
  public:
    virtual bool isHeavy() const;
    virtual ~Book();
};
class Comic : public Book {
    std::string hero;
  public:
    bool isHeavy() const override;
};
```



Calling a virtual method `pb->isHeavy()`:

1. follow `pb` to the object;
2. follow the object's vptr to its class's vtable;
3. fetch the function pointer stored in the `isHeavy` slot (whose index is known at compile time);
4. call it.

This happens at run time, and it selects the right method no matter what the declared type of the pointer is.

29.4 Costs

- **Space**: one vptr (8 bytes) per object, plus one vtable per class.
- **Time**: two extra pointer dereferences per virtual call, and virtual calls usually can't be inlined.

That is why C++ makes methods non-virtual by default: you don't pay for dynamic dispatch unless you ask for it. It is also why a class with no virtual methods has exactly the same layout as a C struct.

29.5 Object Layout and Inheritance

Why does `Book *pb = &someComic;` work at all? The compiler lays out a subclass object as *the superclass part first*, followed by the subclass's own fields (the vptr, if any, comes first of all, as in g++). So a pointer to a Comic is also a valid pointer to its Book part: the Book fields are at the same offsets in both, and code compiled for Book can access them.

This also explains slicing: `Book b = comic;` copies only the first part of the Comic (the part that fits in a Book), and `b`'s vptr is set by Book's constructor to Book's vtable — so `b.isHeavy()` calls `Book::isHeavy`.

29.6 Calling Virtual Methods in Constructors

During Book's constructor (step 2 of constructing a Comic), the vptr points to *Book's* vtable; only once Comic's constructor starts is it switched to Comic's. So a virtual call made inside a base class constructor (or destructor) does *not* dispatch to the subclass override: the subclass part isn't built yet (or has already been destroyed).

Rule

Don't rely on virtual dispatch inside constructors and destructors.

Summary

- Each class with virtual methods has a vtable of method pointers; each object of such a class has a vptr to its class's vtable.
- A virtual call follows object → vptr → vtable slot → code.
- Cost: space for a vptr per object and an indirection per call; hence non-virtual is the default.
- Subclass objects start with the superclass part, which makes upcasting free.
- In ctors/dtors, virtual calls use the class currently being constructed/destroyed.

Chapter 30

Topic: Multiple Inheritance

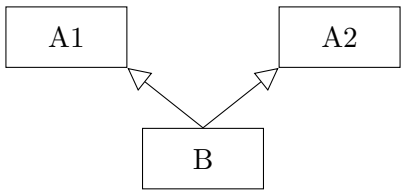
Material from the remainder of the term, not covered in the original lecture notes.

30.1 Basics

A class can inherit from more than one class:

```
class A1 {
  public:
    int a;
};
class A2 {
  public:
    int b;
};
class B : public A1, public A2 {
  public:
    int c;
};

B obj;
obj.a; obj.b; obj.c;    // B has all three fields
```



30.2 Ambiguity

Problems arise when the base classes have members with the same name:

```
class A1 { public: void foo() { cout << "A1::foo"; } };
class A2 { public: void foo() { cout << "A2::foo"; } };
class B : public A1, public A2 {};
```

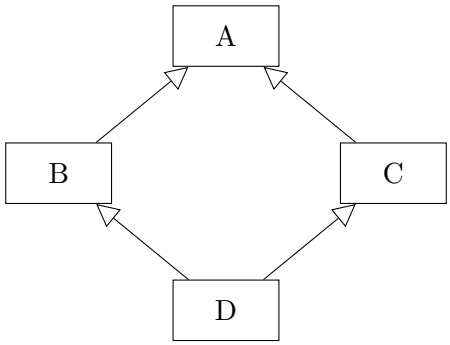
```
B b;
b.foo();       // error: ambiguous
b.A1::foo();   // OK: say which one
b.A2::foo();   // OK
```

30.3 The Diamond Problem

Suppose both bases themselves inherit from a common class:

```
class A { public: int a; };
class B : public A { ... };
class C : public A { ... };
class D : public B, public C { ... };

D d;
d.a;           // error: ambiguous -- which a?
```



By default, a `D` contains *two* A parts: one inside its B part and one inside its C part. So `d.a` is ambiguous; we would have to say `d.B::a` or `d.C::a`. Usually that isn't what we want: there should be only one A in a D.

30.3.1 Virtual Inheritance

Make A a **virtual base class**:

```
class B : virtual public A { ... };
class C : virtual public A { ... };
class D : public B, public C { ... };

D d;
d.a;           // OK: there is only one A part
```

Example from the standard library: `iostream` inherits from both `istream` and `ostream`, which both virtually inherit from `ios_base`/`ios` (the shared stream state).

Construction. With virtual inheritance, B and C must not both construct the single shared A part. The rule is that the *most derived* class constructs the virtual base directly:

```
D::D(...) : A{...}, B{...}, C{...} {}   // D calls A's ctor itself;
                                        // B's and C's calls to A{...} are ignored
```

30.4 Object Layout

Without virtual inheritance, a D looks like

vptr
A fields (B's copy)
B fields
vptr
A fields (C's copy)
C fields
D fields

A pointer to a D works as a pointer to its B part (same start address), but *not* directly as a pointer to its C part, which starts further in. So converting a `D*` to a `C*` must *adjust* the pointer's value:

```
D *pd = new D;
C *pc = pd;                          // compiler adds the offset of the C part
cout << (void*)pd << " " << (void*)pc;   // different addresses!
```

`static_cast` and `dynamic_cast` perform this adjustment; `reinterpret_cast` does not (another reason never to use it for this).

With virtual inheritance, the shared A part is placed at the end of the object, and its location can differ between a standalone B, a B inside a D, etc. The vtable stores the offset to the virtual base so code compiled for B can find its A part at run time:

vptr (B part)
B fields
vptr (C part)
C fields
D fields
vptr
A fields (shared)

This is why accessing members of a virtual base is slightly slower, and why virtual inheritance should not be used unless the diamond actually arises.

30.5 When to Use Multiple Inheritance

- Multiple inheritance of *interfaces* (abstract classes with only pure virtual methods, like Java interfaces) is common and safe: e.g. a class that is both an `Observer` and a `Subject` (a cell in a spreadsheet observes the cells it depends on, and is observed by those that depend on it).
- Multiple inheritance of *implementations*, especially diamonds, gets complicated quickly. Prefer composition when the relationship is not clearly is-a for every base.

Summary

- A class may have several base classes; same-named members are ambiguous and must be qualified (`b.A1::foo()`).

- In a diamond, the common base is duplicated unless the inheritance is `virtual`; the most derived class constructs a virtual base.
- Converting a pointer to a non-first base adjusts its address; virtual bases are located through the vtable.
- Multiple inheritance of interfaces is fine; of implementations, use with care.